

GUILHERME CAMPEZ BELON

**Processamento de Linguagem Natural para Sistemas de
Decisão Baseados em Dados Não Estruturados**

São Paulo
2025

GUILHERME CAMPEZ BELON

**Processamento de Linguagem Natural para Sistemas de
Decisão Baseados em Dados Não Estruturados**

Trabalho apresentado à Escola Politécnica
da Universidade de São Paulo para obten-
ção do Título de Engenheiro Mecatrônico.

Área de Concentração:
Engenharia Mecatrônica

Orientador:
Prof. Dr. Flavio Almeida de Maga-
lhães Cipparrone

São Paulo
2025

Este relatório é apresentado como requisito parcial para obtenção do grau de Engenheiro Mecatrônico na Escola Politécnica da Universidade de São Paulo. É o produto do meu próprio trabalho, exceto onde indicado no texto. O relatório pode ser livremente copiado e distribuído desde que a fonte seja citada

Guilherme Campeç Belon

RESUMO

Este trabalho desenvolve e valida um sistema de decisão automatizado que integra Processamento de Linguagem Natural (PLN) e Aprendizado por Reforço (RL) para o mercado financeiro. A metodologia proposta transforma dados textuais não estruturados, extraídos de notícias, em um índice de sentimento quantitativo por meio de modelos de linguagem pré-treinados. Este índice é, então, incorporado como variável de estado em um ambiente de negociação simulado, onde um agente de RL é treinado para otimizar estratégias de operação. Para avaliar o impacto da informação textual, foi conduzida uma análise comparativa entre uma estratégia baseada apenas em indicadores técnicos e outra que adiciona o índice de sentimento. Os resultados da simulação histórica, realizada em um portfólio diversificado de ativos, revelam que a inclusão do sentimento, embora não apresente uma melhora estatisticamente significativa de forma agregada, gerou um impacto positivo notável no perfil de risco-retorno de ativos específicos, demonstrando o valor da informação qualitativa em contextos pontuais.

Palavras-Chave: Análise de Sentimento Financeiro; Trading Algorítmico; Aprendizado por Reforço Profundo; Sistemas de Decisão Autônomos; Dados Não Estruturados; Engenharia Mecatrônica

LISTA DE FIGURAS

1	Diagrama UML geral do sistema de negociação com aprendizado por reforço.	19
2	Fluxograma do processo de extração de sentimento.	25
3	Fluxo de preparação dos dados para uso em agentes de aprendizado por reforço.	27
4	Interação entre o ambiente de negociação e o agente de trade.	37
5	Comparativo da distribuição de Retorno Acumulado, Sharpe Ratio e Draw-down Máximo para os modelos com e sem sentimento.	44
6	Análise de perfil Risco-Retorno com e sem sentimento.	44
7	Distribuição do Alpha (Retorno Acima do Benchmark) para as estratégias com e sem sentimento.	45
8	Dispersão dos retornos das estratégias em relação ao benchmark (buy and hold).	46
9	Distribuição do impacto da adição de sentimento na performance, medido pela diferença no Sharpe Ratio e no Retorno Acumulado.	47

LISTA DE TABELAS

1	Desempenho da estratégia sem o Índice de Sentimento.	40
2	Desempenho da estratégia com o Índice de Sentimento.	41
3	Estatísticas descritivas dos modelos com e sem sentimento.	43
4	Casos de destaque com base na variação do Sharpe Ratio ao se adicionar o índice de sentimento.	48
5	Resultados do Teste t Pareado entre os modelos com e sem sentimento para as principais métricas de desempenho.	49

SUMÁRIO

1	Introdução	8
1.1	Análise de Requisitos	10
1.2	Considerações Profissionais	13
1.3	Análise do Estado da Arte	14
2	Desenvolvimento do Projeto	17
2.1	Arquitetura do Sistema	19
2.2	Processo de Extração dos Dados	22
2.2.1	Coleta de Dados OHLCV	22
2.2.2	Coleta de Notícias Relacionadas	23
2.2.3	Extração do Conteúdo das Notícias	24
2.3	Processo de Extração do Sentimento das Notícias	25
2.3.1	Extração de Vetores Semânticos com BERT	25
2.3.2	Classificação de Sentimento com FinBERT	25
2.4	Organização dos Dados para o Modelo de Trading	27
2.4.1	Criação dos Indicadores Técnicos	28
2.4.2	Construção do Índice de Sentimento	29
2.5	Ambiente de Aprendizado por Reforço	32
2.6	Treino e Teste dos Modelos	35
2.6.1	Configurações do Treinamento	35
2.6.2	Parâmetros do Modelo e da Política	36
2.6.3	Relatórios e Avaliação de Desempenho	37
3	Resultados Obtidos	38

3.1	Análise de Desempenho das Estratégias	39
3.1.1	Estratégia Baseada em Indicadores Técnicos	39
3.1.2	Estratégia com Adição de Sentimento	41
3.2	Análise Comparativa	43
3.2.1	Desempenho Geral e Perfil Risco-Retorno	43
3.2.2	Análise de Alpha e Comparação com o Benchmark	45
3.2.3	Impacto Direto da Análise de Sentimento	47
3.3	Análise de Significância Estatística	49
3.4	Análise Geral dos Resultados	51
4	Conclusões	53
4.1	Avaliação da Pertinência do Projeto na Engenharia Mecatrônica	54
4.2	Análise Crítica da Solução Desenvolvida	56
4.3	Conclusões Finais do Trabalho	60
	Referências	62
	Apêndice A – Código-Fonte do Projeto	65
	Apêndice B – Exemplo de Notebook Usado no Treinamento	122

1 INTRODUÇÃO

A complexidade crescente dos sistemas de tomada de decisão em engenharia exige a integração de dados de diversas fontes, incluindo informações textuais. Diferentemente dos dados estruturados, que seguem um formato padronizado como tabelas ou séries temporais, os textos são considerados dados não estruturados. Essa classificação se deve à ausência de uma organização fixa que permita seu processamento direto por algoritmos tradicionais. Embora contenham informações ricas e contextuais, esses dados exigem técnicas específicas para sua interpretação. Nesse cenário, o desenvolvimento de sistemas autônomos capazes de compreender e utilizar dados textuais torna-se um desafio central na engenharia, impulsionando a adoção de soluções baseadas em Processamento de Linguagem Natural (PLN).

Diante desse contexto, este projeto tem como objetivo central o desenvolvimento de um modelo de PLN capaz de transformar dados textuais em sinais quantitativos, que possam ser integrados a sistemas de tomada de decisão automatizados. O foco está na criação de uma metodologia robusta para a extração de informações relevantes de textos e sua conversão em indicadores numéricos, aplicáveis à diferentes processos de engenharia. A aplicação escolhida para validar a estrutura é o trading algorítmico no mercado da bolsa norte-americana. No entanto, a abordagem proposta é projetada para ser adaptável a diversos domínios da engenharia em que a interpretação automatizada de textos agregue valor.

A motivação para este trabalho decorre da crescente importância de sistemas autônomos capazes de lidar com grandes volumes de dados não estruturados. Em setores como a gestão de projetos, análise de riscos e otimização de processos, a capacidade de extrair informações úteis a partir de documentos textuais pode resultar em decisões mais rápidas e precisas. No caso específico do mercado financeiro, o uso de PLN para analisar notícias e relatórios apresenta grande potencial na identificação de oportunidades de investimento e na mitigação de riscos operacionais.

Com base nessa motivação, o escopo deste projeto foi delimitado à análise de notícias

financeiras, com o intuito de extrair o sentimento expresso nos textos e avaliar seu impacto em decisões algorítmicas. É importante destacar que o objetivo não é prever preços de ativos nem desenvolver um sistema de negociação completo. A meta é construir uma ferramenta que possibilite investigar como o sentimento textual influencia o desempenho de estratégias automatizadas de tomada de decisão. Para assegurar a reprodutibilidade e a segurança dos resultados, o modelo será testado em um ambiente simulado.

A partir dessa delimitação, a metodologia proposta busca empregar modelos de linguagem pré-treinados para realizar a classificação de sentimento em textos financeiros. Os sinais gerados por essa análise serão integrados a um ambiente de simulação baseado em aprendizado por reforço (utilizando a plataforma `Gymnasium` (FOUNDATION, 2025)), permitindo observar de forma sistemática como essas informações qualitativas impactam a performance de estratégias automatizadas.

Por fim, é importante situar este trabalho dentro do panorama atual do estado da arte em Processamento de Linguagem Natural, que tem sido marcado por avanços significativos nos últimos anos. Modelos como o BERT (DEVLIN et al., 2019), que transforma textos em vetores numéricos que capturam o significado contextual das palavras, e o FinBERT (HUANG et al., 2023), adaptado especificamente para abstração de sentimento no domínio financeiro, demonstram a eficácia de abordagens baseadas em aprendizado profundo para lidar com textos em larga escala. Esses modelos servem como base para o desenvolvimento deste projeto, permitindo uma análise de sentimento precisa e contextualizada. A originalidade da proposta reside na combinação dessas técnicas com aprendizado por reforço, o que promove uma abordagem inovadora para a integração de dados não estruturados em sistemas de decisão automatizada. O objetivo final é oferecer uma ferramenta escalável e aplicável a contextos reais de engenharia, como automação de operações financeiras, contribuindo para decisões mais rápidas, precisas e fundamentadas.

1.1 Análise de Requisitos

Esta seção tem como objetivo especificar os requisitos funcionais e não funcionais do sistema de Processamento de Linguagem Natural (PLN) proposto, com foco na geração de sinais quantitativos derivados de textos financeiros não estruturados. O sistema será utilizado como componente central em um modelo de tomada de decisão automatizada, integrando técnicas avançadas de análise de sentimento com estratégias de aprendizado por reforço em ambientes de simulação de mercado.

A estrutura deve atender às demandas de três perfis de usuários distintos, cujas necessidades convergem em torno de desempenho, flexibilidade e interoperabilidade:

1. Analistas Quantitativos: requerem uma ferramenta capaz de extrair, de maneira sistemática e confiável, métricas de sentimento a partir de textos financeiros (e.g., notícias, relatórios de mercado, transcrições de conferências de resultados). A expectativa é que esses sinais sejam não apenas numericamente estáveis, mas também informativos em relação à dinâmica dos ativos, possibilitando a incorporação direta em modelos estatísticos ou quantitativos de previsão e alocação de portfólio.

2. Pesquisadores em PLN e Aprendizado de Máquina: demandam uma plataforma extensível e modular para experimentação. O sistema deve suportar múltiplas arquiteturas de modelos, bem como permitir o acoplamento com diferentes algoritmos de controle e aprendizado por reforço. A reprodutibilidade, a documentação clara e a compatibilidade com bibliotecas amplamente utilizadas - como `Hugging Face` (Hugging Face, 2025) e `Gymnasium` (FOUNDATION, 2025) - são requisitos centrais.

3. Desenvolvedores de Sistemas de Trading Automatizado: necessitam de um sistema robusto, com baixa latência e fácil integração em estruturas já existentes. Para isso, a arquitetura proposta deve adotar princípios de engenharia de software, como separação de responsabilidades, configuração por meio de arquivos externos e suporte a múltiplos formatos de entrada/saída. Além disso, a ferramenta deve possibilitar o monitoramento contínuo dos sinais gerados e oferecer meios de análise das decisões produzidas.

Assim, com relação aos objetivos dos usuários, estes podem ser resumidos da seguinte forma:

- Extrair automaticamente o sentimento de textos financeiros.
- Transformar esse sentimento em um índice numérico.

- Integrar esse índice em um modelo de decisão algorítmica baseado em aprendizado por reforço.
- Testar o sistema em ambientes simulados.
- Reutilizar o sistema em diferentes domínios.

Para atender a esses objetivos, a plataforma proposta deve possuir os seguintes requisitos funcionais:

- **Coleta de Dados:** Realizar a extração automatizada de textos de fontes financeiras por meio de APIs, de forma eficiente e autônoma.
- **Processamento Textual:** Implementar rotinas para o pré-tratamento dos dados textuais coletados.
- **Classificação de Sentimento:** Utilizar modelos pré-treinados, como BERT (DEVLIN et al., 2019) e FinBERT (HUANG et al., 2023), para classificar o sentimento dos textos, permitindo a incorporação de diferentes modelos de PLN.
- **Geração de Índice Numérico:** Converter as classificações de sentimento em um índice quantitativo que reflita com fidelidade a emoção predominante nos dados.
- **Integração com RL:** Conectar os dados processados e o índice de sentimento a um ambiente de aprendizado por reforço para a tomada de decisão simulada.

Além dos requisitos funcionais, a solução deve atender aos seguintes requisitos não funcionais:

- **Modularidade:** Organizar o código de forma modular para garantir a facilidade de manutenção e a evolução contínua do sistema.
- **Desempenho:** Assegurar um tempo de execução eficiente para viabilizar a análise de grandes volumes de textos em prazos aceitáveis.
- **Reprodutibilidade:** Garantir que os experimentos possam ser reproduzidos para assegurar a confiabilidade e a validade dos resultados.
- **Escalabilidade:** Projetar uma infraestrutura que possa ser adaptada para outros domínios de aplicação, ampliando sua relevância.

O cumprimento desses requisitos visa estabelecer uma estrutura de sistema robusta e eficaz. Acredita-se que a plataforma resultante possa representar um avanço significativo no campo do PLN e da tomada de decisão automatizada, oferecendo uma ferramenta valiosa para analistas, pesquisadores e desenvolvedores de estratégias de automação.

A modularidade e a boa organização do código são aspectos fundamentais para garantir a facilidade de manutenção e a evolução contínua do sistema. Além disso, o tempo de execução precisa ser suficientemente eficiente para viabilizar a análise de grandes volumes de textos dentro de prazos aceitáveis. A reprodutibilidade dos experimentos também se mostra crucial, assegurando a confiabilidade e a validade dos resultados obtidos. Já a capacidade de escalar o sistema para outros domínios amplia a aplicabilidade da infraestrutura desenvolvida, agregando valor e aumentando sua relevância em diferentes contextos.

A coleta automatizada de textos a partir de fontes financeiras, por meio de APIs, constitui um requisito central para o funcionamento do sistema. A plataforma deve ser capaz de estabelecer conexões com diferentes fontes, realizando a extração de dados de forma eficiente e autônoma.

Além disso, a classificação de sentimento, realizada com o auxílio de modelos pré-treinados como BERT (DEVLIN et al., 2019) e FinBERT (HUANG et al., 2023), constitui o núcleo do sistema. A estrutura proposta deve ser capaz de incorporar diferentes modelos de PLN e adaptá-los ao contexto específico do mercado financeiro. Posto isso, destaca-se que a geração de um índice de sentimento em formato numérico viabiliza a quantificação das emoções expressas nos textos analisados, a qual deve refletir com fidelidade o sentimento predominante nos dados coletados.

Por fim, a integração dessas informações em um modelo de aprendizado por reforço, voltado à simulação de decisões, permite avaliar o impacto do sentimento textual nos processos decisórios. A solução desenvolvida precisa ser capaz de conectar os resultados dos modelos com um ambiente de simulação baseado em aprendizado por reforço. Dessa forma, acredita-se que a estrutura proposta possa representar um avanço significativo no campo do PLN e da tomada de decisão automatizada, oferecendo uma ferramenta valiosa para analistas quantitativos, pesquisadores e desenvolvedores de sistemas voltados à automação de estratégias.

1.2 Considerações Profissionais

Este projeto propõe o desenvolvimento de um arcabouço de Processamento de Linguagem Natural (PLN) voltado à tomada de decisões automatizadas a partir de textos financeiros. Durante sua elaboração, foram adotadas diretrizes que asseguram a condução ética e tecnicamente responsável do trabalho, conforme descrito a seguir.

Os dados utilizados no sistema são de domínio público e serão obtidos por meio de APIs de notícias financeiras, como a **Alpha Vantage** (Alpha Vantage Inc., 2025). Dessa forma, garante-se o uso legítimo e transparente das informações, sem restrições legais ou implicações quanto à privacidade.

A rastreabilidade e a clareza metodológica serão asseguradas por meio da documentação completa de todas as etapas do projeto, incluindo coleta e pré-processamento de dados, construção e treinamento dos modelos de PLN, e avaliação de desempenho. Essa documentação incluirá fluxogramas, justificativas técnicas e registros de configuração, facilitando a compreensão e eventual reprodução dos resultados por terceiros.

Além disso, o código-fonte e os dados utilizados serão disponibilizados neste documento de modo que seja possível a reutilização e modificação. Aspectos de interpretabilidade dos modelos também serão analisados, visando compreender a influência de diferentes variáveis textuais nas decisões do sistema.

Por fim, reforça-se o compromisso com a integridade acadêmica, por meio do registro rigoroso de fontes e referências, seguindo as normas da ABNT. Todo o desenvolvimento será pautado por critérios técnicos e científicos, com o objetivo de garantir resultados válidos e úteis para futuras aplicações em sistemas automatizados de apoio à decisão no contexto de engenharia e finanças quantitativas.

1.3 Análise do Estado da Arte

O avanço nas técnicas de Processamento de Linguagem Natural (PLN) tem transformado significativamente a forma como sistemas automatizados interagem com dados textuais. Tradicionalmente, tais sistemas baseavam-se em dados estruturados, organizados em tabelas ou bancos relacionais. No entanto, com o crescimento exponencial de dados não estruturados — como notícias, relatórios e comentários em redes sociais — tornou-se indispensável o desenvolvimento de métodos capazes de extrair informações relevantes desses textos de maneira eficiente.

Nesse cenário, a análise de sentimentos aplicada a textos financeiros tem se destacado como uma abordagem promissora para apoiar a tomada de decisão em ambientes automatizados. Diversas pesquisas demonstram o potencial dessa técnica no mercado financeiro, onde a interpretação rápida e precisa de informações pode representar uma vantagem competitiva substancial (DU et al., 2024).

Inicialmente, os esforços nesse campo concentraram-se em abordagens baseadas em dicionários, que utilizam listas pré-definidas de palavras com conotação positiva ou negativa para inferir o sentimento de um texto. Embora simples, essas técnicas são limitadas por sua incapacidade de lidar com ambiguidade e nuances contextuais (WANG et al., 2013).

Com o surgimento do aprendizado de máquina supervisionado, passaram a ser utilizados modelos treinados a partir de bases de dados anotadas, que mostraram desempenho superior em relação às abordagens baseadas em regras (DEVIKA et al., 2016). Um salto qualitativo ocorreu com a introdução dos **word embeddings**, vetores numéricos que permitem representar palavras em espaços vetoriais contínuos, capturando relações semânticas e possibilitando a construção de modelos mais robustos e generalizáveis (MEDHAT et al., 2014).

A evolução continuou com os modelos baseados em arquiteturas de atenção, em especial os da família **Transformer** (VASWANI et al., 2017). Nesse contexto, o **BERT** (**Bidirectional Encoder Representations from Transformers**) destacou-se por permitir uma contextualização bidirecional das palavras, superando os limites dos modelos anteriores e alcançando resultados expressivos em diversas tarefas de PLN (DEVLIN et al., 2019).

No domínio financeiro, variantes especializadas desses modelos foram desenvolvidas. Um exemplo notável é o **FinBERT**, proposto por Huang et al. (2023), que foi pré-treinado

com textos financeiros como relatórios de análise e notícias de mercado, o que aprimorou seu desempenho na tarefa de classificação de sentimentos nesse setor. Estudos demonstram que o **FinBERT** supera modelos genéricos, especialmente por sua capacidade de interpretar termos técnicos e jargões financeiros (HUANG et al., 2023).

Recentemente, Fatouros et al. (2023) também avaliaram o desempenho do ChatGPT-3.5 na análise de sentimentos em finanças, utilizando técnicas de **zero-shot prompting**. O modelo demonstrou resultados superiores ao **FinBERT** em termos de acurácia e correlação com retornos de mercado, indicando o potencial de modelos de linguagem de uso geral, mesmo em domínios altamente especializados. Além disso, contribuições como a de Du et al. (2024) sistematizam as principais técnicas e aplicações da análise de sentimentos em finanças, com destaque para a previsão de preços, alocação de portfólios e estratégias de negociação automatizada.

Ainda no campo da análise de sentimentos, Wankhade et al. (2022) discutem desafios cruciais como ambiguidade semântica, sarcasmo e multilinguismo — todos fatores que impactam diretamente a acurácia dos modelos de PLN. Por sua vez, Sohangir et al. (2018) testaram arquiteturas profundas como **CNNs** e **LSTMs** em dados da plataforma **StockTwits** (StockTwits, 2025), observando que as **CNNs** obtiveram melhor desempenho na extração de padrões textuais.

No tocante à integração de sinais extraídos de textos em sistemas de decisão automatizados, algumas propostas têm explorado o uso do aprendizado por reforço (RL). Li (2017) apresenta uma revisão abrangente sobre o aprendizado por reforço Profundo (DRL), destacando sua aplicação em áreas como finanças, jogos e robótica. Dentre os algoritmos mais utilizados, o **Proximal Policy Optimization (PPO)**, proposto por Schulman et al. (2017), tem se destacado por sua eficiência e estabilidade, sendo particularmente adequado a ambientes dinâmicos como o mercado financeiro.

Avanços importantes também foram registrados na aplicação de redes neurais recorrentes em algoritmos de **PPO**, como demonstrado por Pleines et al. (2022), que evidenciaram a capacidade de generalização do modelo em ambientes parcialmente observáveis. Na mesma linha, Zong et al. (2024) propuseram o **MacroHFT**, um arcabouço de aprendizado por reforço com memória e sensibilidade contextual voltado à negociação de alta frequência, que mostrou desempenho notável em mercados voláteis como o de criptomoedas.

A revisão de Sahu et al. (2023) fornece uma visão detalhada do uso de técnicas de aprendizado por reforço em finanças quantitativas, cobrindo desde previsão de preços até execução de ordens e modelagem comportamental de mercado. Complementarmente, a

obra de Rao e Jelvis (2022) oferece fundamentos teóricos sólidos em aprendizado por reforço com aplicações práticas especialmente em finanças.

A abordagem proposta por Huang (2018) trata o trading como um jogo de decisão sequencial e emprega o DRQN (Deep Recurrent Q-Network) com técnicas de *action augmentation*, especialmente eficazes em ambientes de alta volatilidade e escassez de dados.

De forma geral, a literatura evidencia uma trajetória clara de evolução — partindo de métodos simples baseados em léxicos até soluções complexas de aprendizado profundo com contextualização semântica. A convergência entre essas técnicas e os sistemas de aprendizado por reforço aponta para o surgimento de uma nova geração de agentes autônomos, capazes de tomar decisões com base não apenas em dados quantitativos, mas também em sinais qualitativos derivados de textos.

Nesse cenário, ferramentas como a *Gymnasium* (FOUNDATION, 2025) têm sido utilizadas para simular estratégias baseadas em sentimentos textuais, permitindo testar o impacto dessas informações na performance de agentes de negociação. Para essas aplicações, APIs como a da *Alpha Vantage* (Alpha Vantage Inc., 2025) fornecem dados financeiros em tempo real, essenciais para ambientes de simulação próximos ao mercado real. Além disso, a plataforma *Hugging Face* (Hugging Face, 2025) consolidou-se como repositório central para modelos de linguagem pré-treinados, oferecendo fácil acesso e ferramentas de aprendizado de máquina.

Diante desse panorama, observa-se uma forte tendência de integração entre PLN, aprendizado por reforço e automação de decisões financeiras, apontando para um futuro onde agentes inteligentes terão maior capacidade interpretativa textual e adaptabilidade a diferentes contextos. Assim, as contribuições analisadas servem de alicerce para este trabalho, que se propõe a desenvolver um sistema de decisão automatizado sensível ao sentimento extraído de notícias financeiras, com validação em ambiente simulado. Ao alinhar-se com as mais recentes tendências da literatura, esta pesquisa busca oferecer uma abordagem inovadora, unindo modelos de linguagem e aprendizado sequencial no contexto da engenharia de sistemas inteligentes.

2 DESENVOLVIMENTO DO PROJETO

Este capítulo apresenta o desenvolvimento do sistema proposto para integração de análise de sentimento com algoritmos de aprendizado por reforço aplicados ao mercado financeiro. São descritos a arquitetura computacional construída, os processos de extração e tratamento dos dados, os métodos utilizados na classificação de sentimentos e a estrutura de treinamento e avaliação dos agentes de negociação. A implementação foi baseada em uma estrutura modular e escalável, permitindo a replicação e adaptação do experimento a outros contextos.

A coleta de dados foi realizada por meio de requisições às APIs públicas da plataforma **Alpha Vantage** (Alpha Vantage Inc., 2025), que fornecem informações financeiras e textuais. Foram utilizados dados históricos no formato OHLCV (Open, High, Low, Close, Volume), com granularidade de 1 minuto. Simultaneamente, foram coletadas notícias associadas aos mesmos períodos e ativos, formando a base textual para a análise de sentimento.

O processamento das notícias inclui a segmentação de seus conteúdos em parágrafos, garantindo granularidade adequada para a análise. Cada parágrafo é convertido em um vetor de **embedding** utilizando o modelo pré-treinado **BERT** (DEVLIN et al., 2019). Em seguida, calcula-se a distância euclidiana entre os **embeddings** e vetores de referência, aproximando contextualmente o conteúdo do texto ao ativo analisado. Esse valor é utilizado para ponderar os parágrafos conforme sua proximidade semântica com o tema tratado.

Para cada parágrafo, realiza-se também a classificação de sentimento utilizando o modelo **FinBERT** (HUANG et al., 2023), especializado em textos financeiros. Os resultados são agregados em uma métrica única por notícia, ponderada pelo inverso da distância semântica, atribuindo maior peso aos parágrafos mais representativos.

As métricas de sentimento extraídas ao longo do tempo são consolidadas em um índice temporal contínuo, que reflete a dinâmica do sentimento do mercado. Esse índice pode

ser utilizado diretamente como variável de entrada em modelos de tomada de decisão. Paralelamente, são calculados diversos indicadores técnicos a partir dos dados OHLCV, como médias móveis, índice de força relativa, bandas de Bollinger, Momentum, entre outros.

Com as séries temporais estruturadas, são treinados dois grupos de modelos de aprendizado por reforço. O primeiro utiliza apenas indicadores técnicos como estados, servindo como linha de base. O segundo grupo combina os mesmos indicadores com o índice de sentimento. Todos os modelos são treinados em ambiente de simulação compatível com a biblioteca `Gymnasium` (FOUNDATION, 2025).

Durante os ciclos de treinamento e teste, registram-se métricas de desempenho dos agentes, como retorno acumulado, índice de Sharpe e percentual de operações lucrativas. Esses dados são organizados em relatórios de performance, permitindo análises comparativas rigorosas entre as diferentes configurações de entrada.

As seções seguintes detalham cada uma das etapas descritas. A Seção 2.1 apresenta a arquitetura do código desenvolvido, com suas classes, métodos e atributos. A Seção 2.2 aborda o processo de extração e organização dos dados brutos. A Seção 2.3 descreve os métodos de análise de sentimento aplicados aos textos. A Seção 2.4 trata da criação dos dados utilizados no treinamento. Na Seção 2.5, é apresentada a estrutura do ambiente de aprendizado por reforço. Por fim, a Seção 2.6 detalha os processos de treinamento, teste e avaliação dos modelos desenvolvidos.

2.1 Arquitetura do Sistema

O sistema desenvolvido é composto por diversos módulos organizados em arquivos Python, cada um responsável por uma etapa específica do pipeline de análise e decisão. A Figura 1 apresenta o diagrama UML do projeto, destacando as principais classes e suas interações. Esse diagrama fornece uma visão geral da arquitetura, permitindo compreender como os diferentes componentes se comunicam para construir o ambiente de simulação e treinar os agentes de aprendizado por reforço.

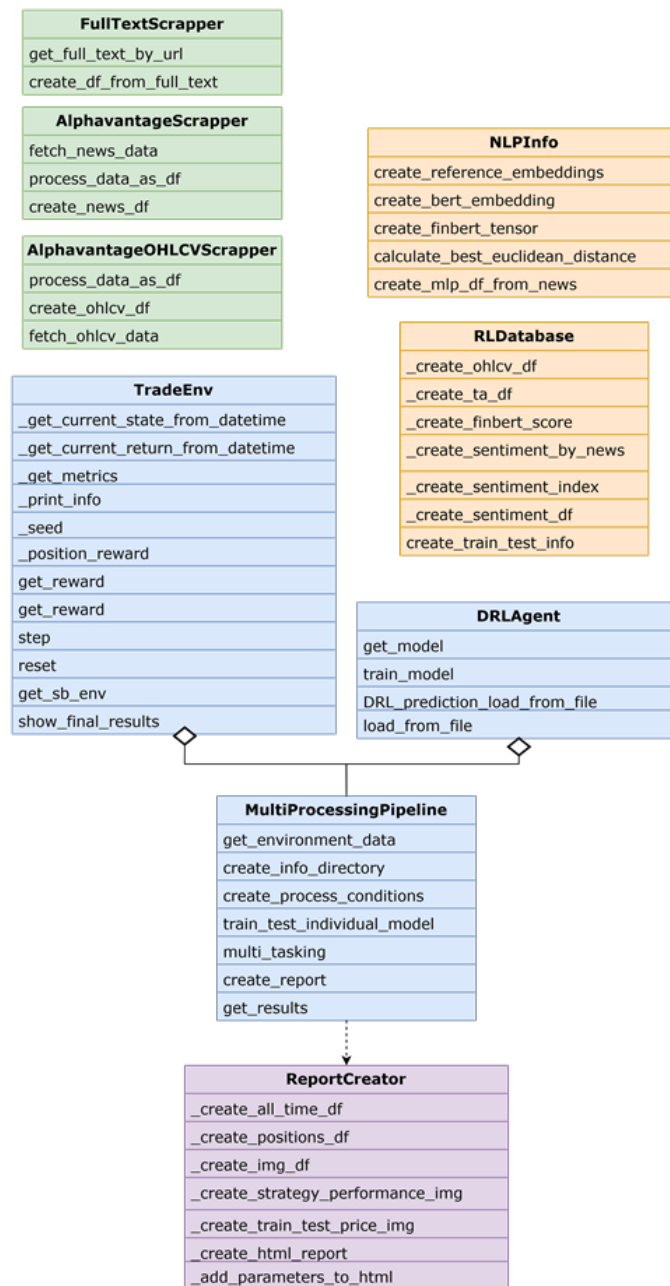


Figura 1: Diagrama UML geral do sistema de negociação com aprendizado por reforço.

As responsabilidades das classes principais são descritas de forma concisa a seguir. Detalhes técnicos sobre os métodos, atributos e a implementação completa de cada componente podem ser consultados no Apêndice A.

2.1.0.1 Módulos de Coleta de Dados

O pipeline inicia-se com três classes dedicadas à extração de dados. A classe denominada `AlphavantageScrapper` coleta notícias financeiras, enquanto os dados de histórico de preços (OHLCV) vêm da `AlphavantageOHLCVScrapper`, ambas a partir da API da Alpha Vantage (Alpha Vantage Inc., 2025). Por fim, a classe `FullTextScrapper` é responsável por extrair o conteúdo textual completo das notícias a partir das URLs fornecidas, utilizando a biblioteca `BeautifulSoup` (RICHARDSON, 2025).

2.1.0.2 Classe NLPInfo (sentiment_index_creator.py)

Esta classe centraliza as tarefas de Processamento de Linguagem Natural (PLN). Ela utiliza modelos pré-treinados como `BERT` (DEVLIN et al., 2019) e `FinBERT` (HUANG et al., 2023) para processar os textos das notícias, gerar vetores semânticos (`embeddings`) e classificar o sentimento. A classe também calcula a distância euclidiana para ponderar a relevância dos textos, gerando como saída uma base de dados estruturada com os scores de sentimento.

2.1.0.3 Classe RLDatabase (rl_db.py)

Atuando como o núcleo de processamento, a `RLDatabase` consolida todas as fontes de informação. Sua função é agregar os dados de mercado (OHLCV), calcular uma série de indicadores técnicos e integrar o índice de sentimento gerado pela classe `NLPInfo`. Ao final, ela normaliza e prepara os conjuntos de dados, dividindo-os em amostras de treino e teste para serem consumidas pelo agente de RL.

2.1.0.4 Classe TradeEnv (trade_env.py)

A classe `TradeEnv` implementa o ambiente de simulação de negociação, seguindo a interface padrão da biblioteca `Gymnasium` (FOUNDATION, 2025). Ela gerencia o estado do agente (posição, saldo), executa as ações de compra e venda, calcula a função de recompensa a cada passo e rastreia todas as métricas de desempenho. O ambiente é projetado para ser flexível, permitindo a configuração de diversos parâmetros de simulação.

2.1.0.5 Classe DRLAgent (DRL_agent.py)

Esta classe funciona como uma interface unificada para os algoritmos de aprendizado por reforço profundo da biblioteca `Stable Baselines3` (RAFFIN et al., 2021). Ela gerencia a configuração do modelo, encapsula os laços de treinamento e os processos de previsão, além de assegurar a reprodutibilidade dos experimentos por meio do controle de sementes de aleatoriedade.

2.1.0.6 Classe MultiProcessingPipeline (multi_process_pipeline.py)

Para otimizar o tempo computacional, a `MultiProcessingPipeline` coordena a execução paralela de múltiplos experimentos de treinamento e teste. Ela é responsável por instanciar e gerenciar os processos, garantindo que cada um opere de forma independente com suas próprias configurações de ambiente, modelo e agente.

2.1.0.7 Classe ReportCreator (report_creator.py)

Ao final da execução, a classe `ReportCreator` consolida os resultados e gera relatórios de desempenho completos. Utilizando a biblioteca `QuantStats` (AROUSSI, 2020), ela produz visualizações gráficas da performance da estratégia, compara os resultados com um benchmark e exporta um relatório interativo em formato HTML, que inclui tanto as métricas quantitativas quanto os parâmetros do experimento.

2.2 Processo de Extração dos Dados

Para viabilizar o treinamento e a avaliação dos agentes de aprendizado por reforço, foi necessário construir um banco de dados contendo informações históricas de mercado e notícias relacionadas a ativos relevantes. Este processo foi dividido em três etapas principais: coleta de dados de mercado (OHLCV), extração de notícias e recuperação do conteúdo textual das matérias jornalísticas.

2.2.1 Coleta de Dados OHLCV

A coleta de dados de mercado foi realizada utilizando a API da **Alpha Vantage** (Alpha Vantage Inc., 2025), com granularidade de 1 minuto, abrangendo o período de 1º de abril a 31 de agosto de 2024. Os dados obtidos incluem os preços de abertura, fechamento, máxima, mínima e volume de negociação para cada ativo em cada intervalo de tempo (OHLCV – Open, High, Low, Close, Volume).

Os ativos monitorados estão listados abaixo, juntamente com as respectivas empresas:

- AAPL – Apple
- AMZN – Amazon
- BA – Boeing
- BAC – Bank of America
- CAT – Caterpillar
- CVX – Chevron
- DE – John Deere
- DIS – Disney
- F – Ford
- GE – General Electric
- GOOG – Google
- GS – Goldman Sachs
- INTC – Intel

- KO – Coca-Cola
- MCD – McDonald's
- META – Meta
- MSFT – Microsoft
- NVDA – NVIDIA
- ORCL – Oracle
- PFE – Pfizer
- TSLA – Tesla
- V – Visa
- XOM – Exxon Mobil

A extração foi realizada por meio do script `ohlcv_scrapper.py`, cuja principal classe é a `AlphavantageOHLCVScraper`. Essa classe realiza requisições à API da **Alpha Vantage** (Alpha Vantage Inc., 2025) utilizando uma chave carregada de variáveis de ambiente, organiza os dados em formato de tabela, renomeia colunas com o ativo correspondente, ajusta os índices temporais e garante a cronologia dos registros.

Por fim, os dados são armazenados em arquivos `.CSV`, formato suficiente para o contexto computacional disponível, embora, urge destacar, que o uso de bancos de dados (SQL ou NoSQL) fosse mais adequado para aplicações em larga escala.

2.2.2 Coleta de Notícias Relacionadas

Complementando os dados de mercado, foram coletadas notícias relacionadas aos ativos monitorados, também utilizando a API da **Alpha Vantage** (Alpha Vantage Inc., 2025) e cobrindo o mesmo intervalo de datas. As principais informações retornadas pela API, em formato `JSON`, que foram utilizadas neste projeto incluem:

- `time`: horário de publicação da notícia;
- `title`: título da notícia;
- `url`: link para o conteúdo completo;

- **summary**: resumo textual;
- **source**: domínio da fonte;
- **ticker**: ativo relacionado;

Esses dados foram processados pelo script `news_scrapper.py`, cuja principal classe, `AlphavantageScrapper`, realiza requisições e paginações robustas, convertendo os dados em tabelas estruturadas e livres de duplicações. Embora as informações de sentimento fornecidas pela API estejam presentes, elas não foram utilizadas neste projeto, uma vez que uma técnica alternativa para análise de sentimento foi adotada conforme descrito posteriormente.

2.2.3 Extração do Conteúdo das Notícias

A API da **Alpha Vantage** (Alpha Vantage Inc., 2025) não fornece o texto completo das notícias, apenas seus links. Para extrair o conteúdo, foi necessário realizar requisições adicionais às URLs fornecidas, obtendo as páginas HTML das matérias. O conteúdo relevante foi extraído utilizando a biblioteca `BeautifulSoup` (RICHARDSON, 2025), buscando especificamente os textos contidos nas tags `<p>`, que representam os parágrafos principais dos artigos.

Essa etapa foi implementada no script `full_text_collector.py`, através da classe `FullTextScrapper`. Seu método `get_full_text_by_url` realiza a coleta individual das páginas, enquanto o método `create_df_from_full_text` aplica o processamento em lote a partir de uma tabela contendo os links. A cada 100 notícias processadas, os resultados intermediários são salvos em disco para evitar perda de dados em caso de interrupção. O resultado final é uma tabela contendo textos brutos, **timestamps** e eventuais falhas, pronto para posterior análise de linguagem natural (NLP) e classificação de sentimento.

2.3 Processo de Extração do Sentimento das Notícias

A classificação do sentimento das notícias é realizada com base no conteúdo completo dos parágrafos obtidos na Seção 2.2. Utilizam-se dois modelos pré-treinados disponibilizados pela biblioteca `transformers` (WOLF et al., 2020), da plataforma Hugging Face (Hugging Face, 2025), conforme o fluxograma ilustrado na Figura 2.

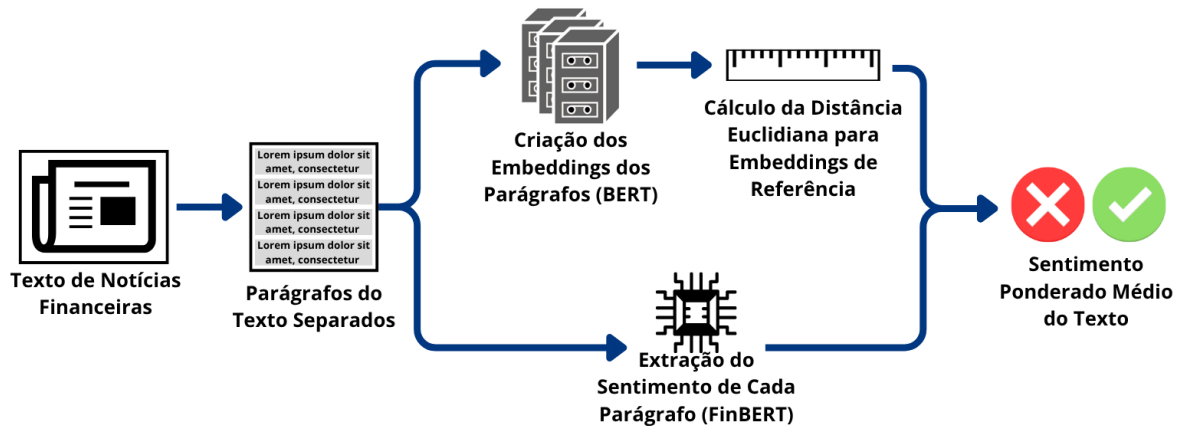


Figura 2: Fluxograma do processo de extração de sentimento.

2.3.1 Extração de Vetores Semânticos com BERT

Nesta etapa, emprega-se o modelo `bert-large-uncased`, desenvolvido pelo Google (DEVLIN et al., 2019). Primeiramente, realiza-se a tokenização do texto dos parágrafos, convertendo cada um em um vetor de entrada compatível com o modelo. A saída do BERT é um vetor de 1024 dimensões que representa a informação semântica contida no texto.

Esse vetor é então comparado, por meio da distância euclidiana, com vetores de referência previamente definidos para cada ativo. A menor distância obtida indica a maior proximidade semântica entre o parágrafo e o conteúdo típico do respectivo ativo. Ao final deste processo, armazena-se somente o valor da menor distância, que será utilizado em etapas posteriores do projeto.

2.3.2 Classificação de Sentimento com FinBERT

A segunda etapa utiliza o modelo `yyanghkust/finbert-tone`, derivado do BERT e ajustado para o domínio financeiro (HUANG et al., 2023). O texto é novamente tokenizado conforme os requisitos do modelo, gerando um vetor de entrada.

A saída do modelo é um vetor de três dimensões, correspondente às probabilidades associadas a cada polaridade de sentimento: neutro, positivo e negativo. Esse vetor é então armazenado para posterior pós-processamento e análise agregada, permitindo avaliar a tendência geral do sentimento ao longo do tempo para cada ativo.

2.4 Organização dos Dados para o Modelo de Trading

A classe `RLDatabase` implementa todo o fluxo de preparação dos dados utilizados pelos agentes de aprendizado por reforço, estruturando informações do mercado financeiro em um formato adequado ao modelo. O processo se inicia com a leitura de séries históricas de preços (OHLCV: Open, High, Low, Close, Volume) a partir de arquivos `.CSV`. Em seguida, são aplicados indicadores técnicos definidos por um dicionário de configuração, por meio de funções internas e da biblioteca `Pandas Technical Analysis` (JOHNSON, 2020), resultando em uma tabela com variáveis descritivas dos preços.

Adicionalmente, podem ser incorporadas métricas de sentimento extraídas de notícias financeiras, geradas pelo modelo `FinBERT` (HUANG et al., 2023) e refinadas por distâncias semânticas calculadas a partir de `embeddings BERT` (DEVLIN et al., 2019). Após ponderações e suavizações temporais, obtém-se um índice contínuo de sentimento. Os dados de preços, indicadores técnicos e, quando habilitado, o índice de sentimento são integrados em uma única tabela, normalizada e segmentada em janelas de treino e teste com sobreposição de contexto.

A estrutura final gera dois conjuntos principais: um contendo apenas os indicadores técnicos e outro que inclui o índice de sentimento, ambos com a mesma interface e organização. A Figura 3 ilustra esse fluxo de preparação de dados realizado pela classe `RLDatabase`.

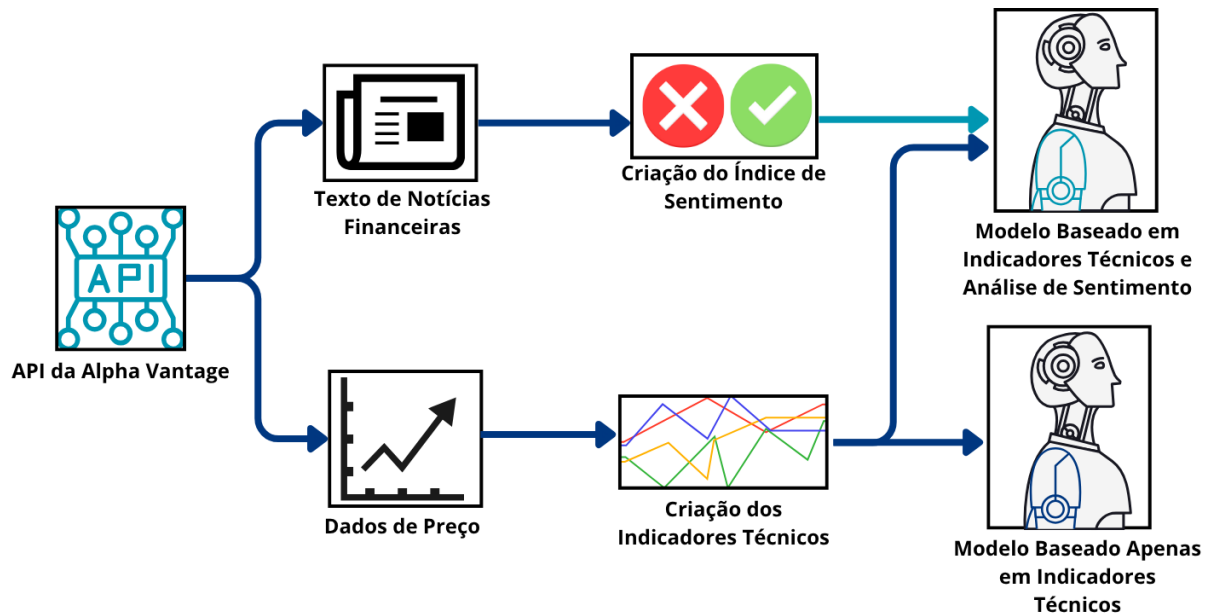


Figura 3: Fluxo de preparação dos dados para uso em agentes de aprendizado por reforço.

2.4.1 Criação dos Indicadores Técnicos

Os indicadores técnicos constituem a base quantitativa do modelo, permitindo capturar padrões de preço, tendência, volatilidade e momentum. A seguir, listam-se os indicadores utilizados, com seus parâmetros e respectivas implementações, que podem ser de desenvolvimento próprio ou oriundas da biblioteca `Pandas Technical Analysis` (JOHNSON, 2020).

- **Índice de Força Relativa Estocástico (Stochastic RSI)**

- Descrição: Mede a posição do RSI atual em relação ao seu intervalo recente.
- Períodos: 14 (RSI), 14 (Estocástico)
- Código: Desenvolvimento próprio

- **Índice de Força Relativa (RSI)**

- Descrição: Avalia a velocidade e mudança dos movimentos de preço.
- Período: 14
- Código: Desenvolvimento próprio

- **Average True Range (ATR)**

- Descrição: Mede a volatilidade média dos preços.
- Período: 14
- Código: `pandas_ta.atr`

- **Momentum**

- Descrição: Indica a velocidade da variação de preço.
- Período: 14
- Código: `pandas_ta.mom`

- **Commodity Channel Index (CCI)**

- Descrição: Mede o desvio do preço em relação à média.
- Período: 14
- Código: `pandas_ta.cci`

- **Oscilador Estocástico (Stochastic)**

- Descrição: Compara o preço de fechamento com a faixa de preços no período.
- Período: 14
- Código: `pandas_ta.stoch`

- **Moving Average Convergence Divergence (MACD)**

- Descrição: Avalia convergência e divergência de médias móveis.
- Períodos: 12 (rápida), 26 (lenta), 9 (sinal)
- Código: `pandas_ta.macd`

- **Volume**

- Descrição: Mede o volume negociado e sua variação relativa.
- Período: 14
- Código: Desenvolvimento próprio

- **Bandas de Bollinger**

- Descrição: Faixa de desvio padrão em torno de uma média móvel.
- Período: 20
- Código: Desenvolvimento próprio

2.4.2 Construção do Índice de Sentimento

A seguir, apresenta-se o processo de geração do índice de sentimento, implementado na classe `RLDatabase`, com base nas distâncias semânticas e scores fornecidos pelo modelo FinBERT (HUANG et al., 2023).

Para cada notícia i , o modelo retorna um vetor de probabilidades:

$$\mathbf{f}_i = \begin{bmatrix} f_i^{(\text{neutro})} \\ f_i^{(\text{positivo})} \\ f_i^{(\text{negativo})} \end{bmatrix}. \quad (2.1)$$

Define-se o score bruto como a diferença entre as componentes positiva e negativa:

$$s_i = f_i^{(\text{positivo})} - f_i^{(\text{negativo})}. \quad (2.2)$$

2.4.2.1 Agregação por Distância Semântica

Obtida a menor distância semântica d_i entre o vetor BERT da notícia e os vetores de referência do ativo, pondera-se cada notícia pelo seu inverso:

$$w_i = \frac{1}{d_i}. \quad (2.3)$$

Em seguida, para cada instante t , agregam-se os scores ponderados das notícias publicadas nesse horário:

$$S_t = \frac{\sum_{i: \text{data}(i)=t} w_i s_i}{\sum_{i: \text{data}(i)=t} w_i}. \quad (2.4)$$

2.4.2.2 Construção Temporal com Decaimento Exponencial

Define-se uma grade temporal t_k com frequência de 1 minuto entre abertura e fechamento do mercado. Com uma janela de lookback $\Delta = 1$ dia, define-se:

$$\tau_{k,i} = (t_k - t_i)_+, \quad \text{com } \tau_{k,i} \leq \Delta, \quad (2.5)$$

sendo t_i os instantes das notícias com valores $\tilde{S}_{t_i}$. A ponderação exponencial é:

$$\omega_{k,i} = \exp\left(-\frac{\tau_{k,i}}{\Delta}\right). \quad (2.6)$$

Para facilitar a comparação temporal, os valores de S_t são normalizados pelo valor absoluto máximo dentro da janela Δ definida:

$$\tilde{S}_t = \frac{S_t}{\max_t |S_t|_\Delta}. \quad (2.7)$$

Se houver pelo menos uma notícia no intervalo:

$$I(t_k) = \frac{\sum_{i: t_k - \Delta < t_i \leq t_k} \omega_{k,i} \tilde{S}_{t_i}}{\sum_{i: t_k - \Delta < t_i \leq t_k} \omega_{k,i}}. \quad (2.8)$$

Caso contrário, aplica-se decaimento ao valor anterior:

$$I(t_k) = 0.9 \times I(t_{k-1}). \quad (2.9)$$

O índice contínuo é então interpolado e alinhado à grade temporal de preços, integrando-se como variável adicional na tabela final usada pelo modelo de aprendizado por reforço.

2.5 Ambiente de Aprendizado por Reforço

Ambientes de aprendizado por reforço são formulados como um processo de decisão sequencial, no qual um agente interage com um ambiente ao longo do tempo com o objetivo de maximizar uma função de recompensa cumulativa. A cada passo de tempo t , o agente observa um estado s_t , escolhe uma ação a_t a partir de um espaço de ações $\mathcal{A}$, e recebe uma recompensa r_t , além de transitar para um novo estado s_{t+1} , definido por uma função de transição de estados. Os principais componentes desses ambientes são:

- **Estado** (s_t): representação informacional do ambiente em determinado instante;
- **Ação** (a_t): decisão tomada pelo agente, podendo ser discreta ou contínua;
- **Recompensa** (r_t): valor escalar fornecido ao agente como resposta à ação tomada;
- **Função de Transição**: regra que determina a evolução do ambiente;
- **Reset**: procedimento que reinicia o ambiente, devolvendo-o ao estado inicial, utilizado ao término de um episódio.

Com base nessa estrutura, implementa-se um ambiente de negociação para séries temporais financeiras, no qual o agente é treinado para interagir com dados de mercado simulando decisões de compra, venda ou manutenção de posição. O ambiente é desenhado para ser compatível com algoritmos de RL modernos e oferece suporte para avaliação robusta de políticas de negociação.

Neste ambiente, o estado observado pelo agente em cada passo consiste em uma matriz contendo variáveis técnicas extraídas dos N últimos instantes da série temporal — tais como preços, indicadores de momentum, medidas de volatilidade e outros fatores técnicos. Essa matriz possui dimensão $N \times F$, onde F representa o número de atributos. A esse estado soma-se um vetor adicional com duas informações: o tipo de posição atualmente em aberto (comprada, vendida ou neutra) e o retorno acumulado dessa posição até o instante corrente. Essa modelagem garante que o agente tenha visibilidade tanto sobre as dinâmicas recentes do mercado quanto sobre seu estado interno de risco e desempenho.

O espaço de ações é discreto e com três opções: -1 (abertura de posição vendida ou fechamento de posição comprada), 0 (manutenção da posição) e 1 (abertura de posição comprada ou fechamento de posição vendida). Quando o agente seleciona uma ação que altera o estado da posição, o ambiente simula a execução de uma ordem de mercado no timestamp subsequente.

Para fins de simplificação e compatibilidade com diversos trabalhos da literatura, adotaram-se duas aproximações usuais nesse tipo de simulação: (i) ausência de custos de transação e (ii) execução imediata da ordem no próximo instante temporal. Essas hipóteses visam isolar e avaliar a efetividade da política de decisão aprendida pelo agente, eliminando variáveis externas que podem mascarar o desempenho real do algoritmo. Trabalhos recentes como os de Li et al. (2019) e Pardo et al. (2022) também assumem execução perfeita e zero fricção, especialmente em fases preliminares de desenvolvimento de ambientes de aprendizado por reforço em finanças, como forma de reduzir a complexidade computacional e facilitar a convergência da política.

A execução no timestamp seguinte é particularmente comum em abordagens com discretização temporal de alta frequência (por exemplo, minuto a minuto), em que a latência é assumida desprezível frente à granularidade do dado histórico disponível e à liquidez dos ativos simulados. Além disso, a exclusão inicial de custos operacionais permite comparar diferentes arquiteturas e algoritmos sob condições padronizadas, antes de introduzir penalidades mais realistas. Estudos como Richards (2023) e Gasperov e Kostanjcar (2021) seguem essa linha ao construir ambientes de simulação que priorizam robustez estrutural e replicabilidade experimental.

A dinâmica temporal do ambiente é governada por um cursor que avança um passo de 1 minuto a cada iteração. O retorno do período corrente é calculado como a variação percentual entre o preço atual e o anterior, sendo este multiplicado pela direção da posição (positiva para longas, negativa para curtas, nula para neutras) e incorporado à riqueza simulada do agente. Ao fechar uma posição, o ambiente registra um resumo da operação, contendo data e hora de entrada, saída, tipo e retorno obtido, e o agente retorna ao estado neutro.

Durante a simulação, o ambiente mantém uma lista de retornos individuais para fins de cálculo de recompensa. Existem diferentes estratégias de recompensa configuráveis, tais como: retorno da iteração atual, retorno acumulado da posição em aberto, ou ainda estimativas de retorno futuro a determinado horizonte. Também é possível aplicar transformações logarítmicas ao retorno. Essa flexibilidade permite alinhar a recompensa aos objetivos específicos da política — seja maximização de lucro imediato, consistência ao longo do tempo ou antecipação de tendências de curto prazo.

O episódio é encerrado quando o cursor percorre todo o conjunto de dados históricos, momento em que o ambiente sinaliza o fim do episódio e pode emitir um relatório consolidado com as métricas de performance: retorno total, índice de Sharpe, Drawdown

Máximo, taxa de acerto, entre outras. Gráficos de evolução do capital e tabelas com as posições tomadas também são gerados, além da possibilidade de exportação de um relatório HTML completo, o que facilita a análise quantitativa e qualitativa da estratégia.

O comando `reset` reinicia o ambiente, restaurando o capital inicial, limpando listas internas e zerando todas as posições. A leitura dos dados é retomada a partir da janela inicial, garantindo que o primeiro estado disponibilizado ao agente contenha um histórico suficiente para avaliação contextual. A inicialização do ambiente pode utilizar uma semente fixa para garantir reprodutibilidade dos experimentos.

Por fim, destaca-se que o projeto do ambiente é modular e compatível com bibliotecas modernas de RL que utilizam vetorização, permitindo simulações paralelas e treinamento com diversos algoritmos. Em síntese, o ambiente implementado oferece um fluxo completo de simulação: ingestão de dados financeiros históricos, formação de estados ricos em contexto temporal e de risco, execução de ordens, retroalimentação através de recompensas personalizáveis e consolidação de métricas para avaliação robusta de políticas de negociação aprendidas via aprendizado por reforço.

2.6 Treino e Teste dos Modelos

O uso de algoritmos de aprendizado por reforço em operações algorítmicas tem ganhado destaque pela capacidade de desenvolver políticas adaptativas em ambientes altamente dinâmicos. Neste trabalho, adota-se o algoritmo **Recurrent Proximal Policy Optimization (RPP0)**, disponível na biblioteca **sb3-contrib** do **Stable Baselines3** (RAFFIN et al., 2021), por sua robustez frente a características típicas dos mercados financeiros, como ruído, não-estacionariedade e dependência temporal parcial.

O RPP0 é uma extensão do **Proximal Policy Optimization (PPO)**, originalmente proposto por Schulman et al. (2017), que incorpora estruturas recorrentes como redes **LSTM (Long Short-Term Memory)**. Essas redes permitem ao agente manter uma memória dos estados anteriores, essencial para capturar padrões temporais complexos, como reversões e mudanças de regime. Diferentemente de abordagens baseadas em estados independentes, o RPP0 infere estados ocultos a partir de observações parciais, sendo particularmente eficaz em ambientes com múltiplos ativos e variáveis quantitativas.

Além das vantagens da estrutura recorrente, o RPP0 preserva os principais benefícios do PPO, como estabilidade durante o treinamento. A capacidade de adaptação do algoritmo a diferentes regimes de mercado é destacada por Pleines et al. (2022), que evidencia sua resiliência e sensibilidade temporal.

2.6.1 Configurações do Treinamento

O treinamento dos modelos utiliza dados históricos com janela de treino de 01 de abril de 2024 a 20 de julho de 2024 e período de teste até 31 de agosto de 2024. O ambiente simula operações com posições compradas e vendidas, refletindo a flexibilidade das estratégias quantitativas.

A recompensa adotada é baseada no logaritmo do retorno da posição, escalado por um fator de 10^4 , o que contribui para estabilizar o treinamento e melhorar o gradiente. O estado de entrada do agente é composto por uma janela de 60 períodos (equivalente a uma hora, dado o intervalo de execução de ordens por minuto). A principal diferença entre os dois modelos treinados está na presença ou não do índice de sentimento como variável adicional no estado.

2.6.2 Parâmetros do Modelo e da Política

Os principais hiperparâmetros utilizados para o RPP0 são:

- `n_steps`: 60
- `gamma`: 0,995
- `ent_coef`: 0,005
- `learning_rate`: 0,0003
- `batch_size`: 60

A arquitetura da política recorrente é definida por meio do argumento `policy_kwargs`, combinando camadas densas e estruturas LSTM. A rede se divide em duas subestruturas: uma voltada à estimativa da função de valor (`vf`) e outra à política de ação (`pi`).

A parte da função de valor é composta por três camadas densas com 128 neurônios cada, responsáveis por mapear os estados para estimativas escalares de retorno. A subestrutura da política, por sua vez, contém duas camadas densas com 128 neurônios cada, responsáveis por determinar a distribuição de probabilidade sobre as ações possíveis.

Complementando essas estruturas, são adicionadas duas camadas LSTM com 256 unidades ocultas cada. Essas redes são adequadas para capturar dependências temporais de longo prazo, mantendo memória interna que evolui com a série. Essa combinação permite modelar tanto relações instantâneas quanto padrões temporais sutis e não-locais, comuns no contexto financeiro.

Cada sequência de dados de treino passa por 3 iterações completas, promovendo estabilidade no aprendizado. Esse processo é executado separadamente nos dois cenários — com e sem o índice de sentimento — cada qual implementado em um notebook Jupyter distinto. Utiliza-se a classe `MultiProcessingPipeline` para paralelização dos treinamentos em múltiplas `threads`.

No total, treinam-se 6 modelos em paralelo (3 com sentimento e 3 sem), com tempo médio de execução entre 6 e 7 horas para todo o processo, incluindo aplicação nos dados de teste. No fim, o modelo com melhor desempenho em termos de retorno acumulado no período de teste foi tomado como referência para análise comparativa dos resultados.

A Figura 4 ilustra a interação entre o agente de trade e o ambiente simulado, destacando os fluxos de observação e ação que compõem o ciclo de decisão.

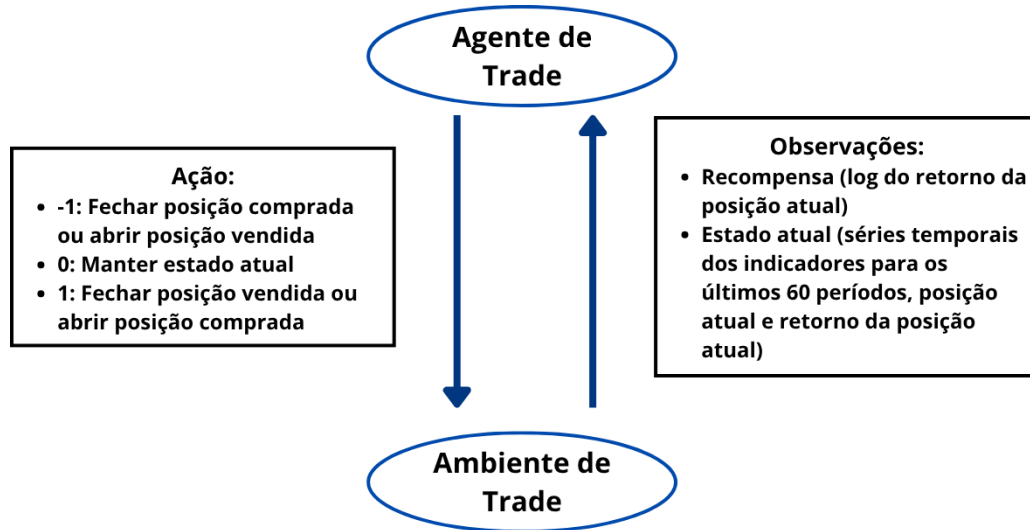


Figura 4: Interação entre o ambiente de negociação e o agente de trade.

2.6.3 Relatórios e Avaliação de Desempenho

Após o treinamento, cada modelo gera um relatório em formato HTML por meio da biblioteca `Quantstats` (AROUSSI, 2020), com comparação direta ao ativo usado no período de teste. Esses relatórios apresentam métricas quantitativas e gráficos relevantes para avaliação do desempenho, incluindo:

- **Sharpe Ratio:** retorno ajustado ao risco;
- **R^2 / Correlação:** associação estatística com o benchmark;
- **Volatilidade:** variação dos retornos;
- **Drawdown Máximo:** maior perda acumulada desde o pico;
- **Retorno Acumulado:** crescimento total do capital;
- **Distribuição de Retornos:** frequência e magnitude de ganhos/perdas;
- **Rolling Sharpe:** evolução do índice Sharpe ao longo do tempo;
- **Comparação com Benchmark:** análise de desempenho relativo;
- **Sequências de Ganhos/Perdas:** padrões de acertos consecutivos.

Esses relatórios viabilizam uma comparação direta entre os modelos com e sem o índice de sentimento, permitindo avaliar objetivamente o impacto da variável adicional na performance das políticas aprendidas.

3 RESULTADOS OBTIDOS

Este capítulo dedica-se à apresentação e análise detalhada dos resultados empíricos obtidos a partir da simulação das estratégias de negociação algorítmica desenvolvidas. O foco central é avaliar o desempenho dos agentes de Aprendizado por Reforço (RL) em um ambiente de backtesting, comparando uma estratégia de base, que utiliza exclusivamente indicadores técnicos, com uma estratégia aprimorada, que incorpora um índice de sentimento derivado de dados textuais não estruturados. A avaliação quantitativa busca mensurar o impacto da inclusão de informações de Processamento de Linguagem Natural (PLN) na performance e no perfil de risco das decisões automatizadas.

A metodologia de avaliação, conforme delineada no Capítulo 2, resultou na geração de um conjunto robusto de métricas de desempenho para um portfólio diversificado de 23 ativos. Para cada ativo, foram treinados e testados dois modelos distintos, permitindo uma comparação direta e pareada. O primeiro modelo serve como linha de base, representando uma abordagem puramente quantitativa tradicional. O segundo modelo, por sua vez, testa a hipótese central deste trabalho: a de que sinais qualitativos extraídos de notícias financeiras podem agregar valor informacional e conduzir a políticas de negociação mais eficazes.

A estrutura deste capítulo inicia-se com uma análise descritiva individual do desempenho de cada uma das duas estratégias, detalhando seus resultados em todo o conjunto de ativos. Em seguida, realiza-se uma análise comparativa aprofundada, focando no perfil risco-retorno, na geração de alfa em relação ao benchmark e no impacto direto da variável de sentimento. Por fim, é conduzida uma análise de significância estatística para validar se as diferenças de desempenho observadas são estatisticamente relevantes, culminando em uma síntese dos principais resultados.

3.1 Análise de Desempenho das Estratégias

Nesta seção, são apresentados os resultados detalhados de desempenho para as duas estratégias de negociação avaliadas: uma baseada exclusivamente em indicadores técnicos e outra que incorpora um índice de sentimento derivado de notícias financeiras. A análise é conduzida de forma individualizada para cada abordagem, possibilitando uma compreensão isolada da performance de cada modelo antes da comparação cruzada. As métricas consideradas incluem retorno da estratégia, retorno do ativo via **buy and hold**, Sharpe Ratio, Drawdown Máximo, e os melhores e piores dias de operação.

3.1.1 Estratégia Baseada em Indicadores Técnicos

A estratégia de base, denominada modelo técnico, utiliza exclusivamente indicadores derivados de séries de preço e volume (OHLCV) como variáveis de entrada no processo de decisão. Essa configuração serve como linha de base para quantificação dos efeitos da adição de variáveis qualitativas, como o sentimento extraído de textos. A Tabela 1 apresenta os resultados de desempenho para os 23 ativos testados.

Ticker	Ret. Estratégia	Ret. Original	Sharpe	Max DD	Best Day	Worst Day
AAPL	12.48%	2.33%	0.12	-14.33%	14.58%	-12.73%
AMZN	4.79%	-2.34%	0.07	-13.14%	8.66%	-7.92%
BA	4.93%	-3.38%	0.08	-14.99%	10.26%	-9.45%
BAC	33.46%	-2.65%	0.33	-6.89%	7.61%	-5.92%
CAT	17.16%	2.20%	0.25	-10.36%	4.64%	-4.24%
CVX	4.45%	-6.04%	0.08	-12.89%	12.53%	-11.12%
DE	11.97%	1.88%	0.18	-14.67%	4.96%	-4.56%
DIS	10.03%	-5.24%	0.12	-23.48%	6.98%	-21.42%
F	8.15%	-5.32%	0.10	-17.43%	11.52%	-10.33%
GE	-6.42%	9.44%	-0.01	-16.41%	6.93%	-7.36%
GOOG	26.73%	-8.03%	0.21	-13.68%	7.51%	-7.09%
GS	-11.05%	6.00%	-0.10	-18.66%	7.84%	-8.66%
INTC	14.53%	-32.69%	0.11	-31.83%	45.78%	-31.37%
KO	10.17%	10.89%	0.19	-3.40%	2.62%	-2.55%
MCD	-9.99%	12.27%	-0.08	-17.55%	7.41%	-7.45%
META	18.31%	9.31%	0.18	-28.33%	7.30%	-9.10%
MSFT	-13.80%	-4.30%	-0.10	-22.85%	6.42%	-7.42%
NVDA	-99.59%	-49.91%	-0.33	-99.62%	50.42%	-77.31%
ORCL	-4.11%	2.05%	-0.04	-15.13%	5.19%	-4.75%
PFE	-3.43%	-1.88%	0.00	-11.82%	10.86%	-9.80%
TSLA	-3.37%	-11.84%	0.00	-22.41%	9.63%	-13.77%
V	14.25%	4.23%	0.17	-9.67%	6.90%	-6.49%
XOM	-3.50%	2.44%	-0.02	-9.92%	6.14%	-5.71%

Tabela 1: Desempenho da estratégia sem o Índice de Sentimento.

Os resultados indicam uma expressiva variabilidade na performance entre os ativos. O modelo técnico apresentou bom desempenho em papéis como Bank of America (BAC), Google (GOOG) e Caterpillar (CAT), cujos retornos superaram significativamente o **buy and hold**. Notam-se também Sharpe Ratios positivos nesses casos, sugerindo que o retorno foi obtido com risco relativamente controlado.

Por outro lado, ativos como NVIDIA (NVDA), Goldman Sachs (GS) e Microsoft (MSFT) apresentaram desempenho negativo. O destaque é NVDA, cujo resultado extremamente desfavorável e drawdown próximo de 100% indicam um colapso completo da estratégia. Esses resultados sugerem que, embora a abordagem técnica consiga identificar padrões lucrativos em certos contextos, sua eficácia não é generalizável entre todos os ativos — possivelmente devido à sensibilidade a ruído de mercado ou regimes de volatilidade distintos.

3.1.2 Estratégia com Adição de Sentimento

A segunda estratégia inclui o índice de sentimento, gerado a partir da análise de textos financeiros por modelos de PLN. Este índice é incorporado ao espaço de estados do agente de aprendizado por reforço como uma variável adicional, permitindo que o agente integre informações contextuais não estruturadas na tomada de decisão. A Tabela 2 exibe os resultados para o mesmo conjunto de ativos.

Ticker	Ret. Estratégia	Ret. Original	Sharpe	Max DD	Best Day	Worst Day
AAPL	-3.31%	2.33%	-0.07	-8.33%	4.82%	-5.02%
AMZN	-10.84%	-2.34%	-0.03	-21.64%	7.92%	-8.66%
BA	33.16%	-3.38%	0.25	-14.29%	9.45%	-10.26%
BAC	21.53%	-2.65%	0.22	-12.28%	7.61%	-8.64%
CAT	24.46%	2.20%	0.35	-9.37%	4.24%	-4.64%
CVX	-8.73%	-6.04%	-0.12	-11.65%	6.78%	-6.96%
DE	7.52%	1.88%	0.15	-12.17%	4.56%	-4.48%
DIS	-5.78%	-5.24%	0.05	-23.77%	24.61%	-19.75%
F	54.46%	-5.32%	0.38	-25.27%	11.52%	-10.33%
GE	-15.84%	9.44%	-0.09	-19.91%	6.83%	-7.36%
GOOG	2.63%	-8.03%	0.05	-9.76%	7.09%	-7.27%
GS	18.28%	6.00%	0.22	-14.39%	8.66%	-7.84%
INTC	18.36%	-32.69%	0.22	-11.96%	10.61%	-11.91%
KO	54.37%	10.89%	0.91	-2.56%	2.62%	-2.55%
MCD	-7.66%	12.27%	-0.06	-16.41%	7.20%	-7.45%
META	6.45%	9.31%	0.08	-16.29%	8.37%	-9.10%
MSFT	-2.78%	-4.30%	-0.00	-16.13%	7.45%	-6.97%
NVDA	247.10%	-49.91%	0.33	-50.42%	101.71%	-50.42%
ORCL	-3.36%	2.05%	0.01	-25.59%	13.70%	-16.58%
PFE	4.30%	-1.88%	0.07	-13.00%	9.80%	-10.86%
TSLA	-14.85%	-11.84%	-0.15	-25.92%	7.13%	-6.66%
V	10.18%	4.23%	0.13	-9.48%	6.90%	-6.49%
XOM	21.80%	2.44%	0.34	-6.14%	5.71%	-6.14%

Tabela 2: Desempenho da estratégia com o Índice de Sentimento.

Com a adição do índice de sentimento, observam-se casos de melhoria expressiva, como nos ativos Ford (F), Coca-Cola (KO) e NVDA. Em especial, destaca-se o retorno expressivo para NVDA, revertendo completamente o colapso observado na estratégia técnica e resultando em um Sharpe Ratio positivo. Esse resultado sugere que o modelo passou a capturar sinais qualitativos não triviais, possivelmente antecipando reações do mercado a eventos noticiosos.

No entanto, a estratégia não apresentou melhorias sistemáticas para todos os ativos.

A Apple (AAPL) e a Amazon (AMZN), por exemplo, registraram desempenho inferior ao modelo técnico. Isso indica que a adição do sentimento pode beneficiar alguns ativos, especialmente aqueles mais sensíveis à percepção do mercado, mas também pode introduzir ruído quando a notícia não possui correlação significativa com o comportamento de preço.

De maneira geral, os Sharpe Ratios foram, em média, mais elevados nesta abordagem, e o número de ativos com retornos positivos aumentou. Além disso, observa-se uma redução nos Drawdowns Máximos em muitos casos, indicando uma possível melhoria na gestão de risco por parte do agente.

3.2 Análise Comparativa

Após a apresentação individual do desempenho de cada estratégia, esta seção realiza uma análise comparativa direta entre o modelo de base e o modelo enriquecido com sentimento. O objetivo é isolar e quantificar o efeito da adição do índice de sentimento nas principais métricas de performance, no perfil de risco-retorno e na capacidade de gerar retornos excedentes em relação ao benchmark. A comparação é feita tanto de forma agregada, considerando o portfólio completo, quanto por meio da análise de casos específicos.

3.2.1 Desempenho Geral e Perfil Risco-Retorno

A primeira etapa da análise comparativa consiste na avaliação das estatísticas descritivas agregadas para o conjunto de ativos. A Tabela 3 resume as métricas centrais de tendência e dispersão para ambos os modelos.

	Ret. Normal	Ret. Sentimento	Sharpe Normal	Sharpe Sentimento	Max DD Normal	Max DD Sentimento
mean	0.02	0.20	0.07	0.14	-0.20	-0.16
std	0.25	0.53	0.14	0.23	0.19	0.10
min	-1.00	-0.16	-0.33	-0.15	-1.00	-0.50
25%	-0.04	-0.05	-0.02	-0.02	-0.21	-0.21
50% (mediana)	0.05	0.06	0.08	0.08	-0.15	-0.14
75%	0.13	0.22	0.18	0.24	-0.12	-0.11
max	0.33	2.47	0.33	0.91	-0.03	-0.03

Tabela 3: Estatísticas descritivas dos modelos com e sem sentimento.

Conforme os dados da Tabela 3, o modelo com sentimento apresentou valores médios superiores tanto para o retorno quanto para o Sharpe Ratio. Além disso, o Drawdown Máximo médio foi ligeiramente menor nesse modelo, sugerindo uma melhor performance geral ajustada ao risco.

No entanto, é importante observar que o desvio padrão do modelo com sentimento foi consideravelmente mais elevado, especialmente para o retorno, indicando maior variabilidade entre os ativos. Esse comportamento é compatível com a presença de ativos com desempenho muito superior à média, o que também se reflete na grande diferença entre a média e a mediana dos retornos. Isso reforça a hipótese de que os resultados superiores do modelo com sentimento são influenciados por outliers de alta performance, enquanto a maior parte dos ativos apresenta resultados comparáveis entre os dois modelos.

A visualização dessas distribuições por meio de diagramas de caixa, apresentada na Figura 5, corrobora a análise.

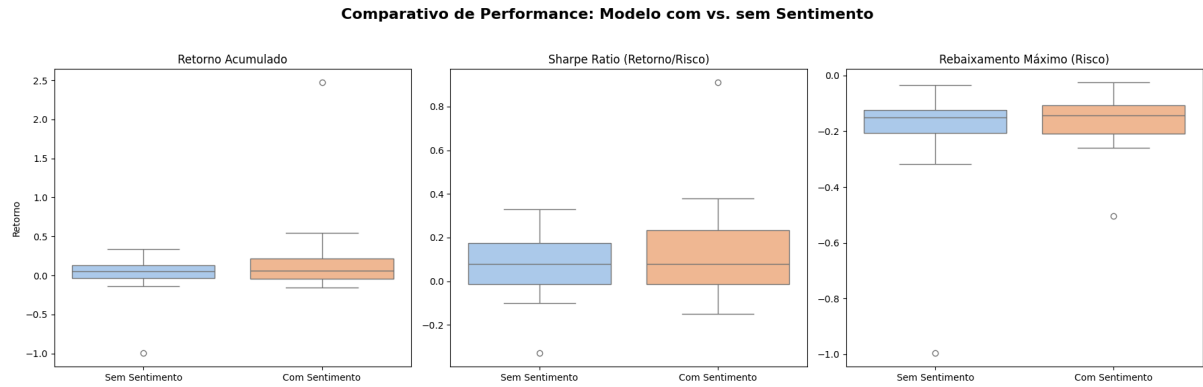


Figura 5: Comparativo da distribuição de Retorno Acumulado, Sharpe Ratio e Drawdown Máximo para os modelos com e sem sentimento.

Os diagramas de caixa confirmam que, embora as medianas sejam próximas, a estratégia com sentimento exibe uma dispersão interquartil muito maior e a presença de outliers positivos proeminentes, o que explica a média mais alta e o desvio padrão elevado. Para aprofundar a análise da relação entre risco e retorno, a Figura 6 dispõe cada ativo em um gráfico de dispersão, onde o eixo vertical representa o retorno acumulado e o eixo horizontal, o risco (medido como o inverso do Drawdown Máximo).

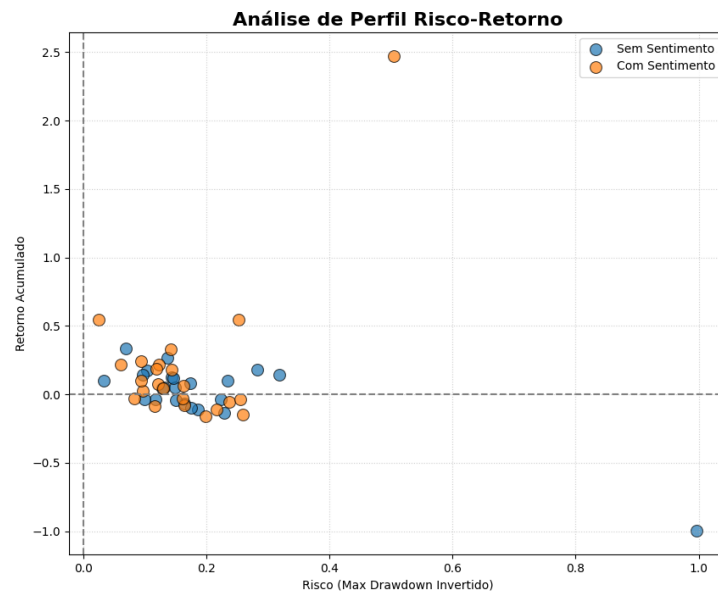


Figura 6: Análise de perfil Risco-Retorno com e sem sentimento.

O perfil de risco-retorno ilustra que a adição de sentimento moveu os ativos em diferentes direções no plano risco-retorno. Enquanto alguns ativos (pontos laranjas) migraram para o quadrante superior esquerdo (maior retorno, menor risco), outros tiveram seu perfil

deteriorado. Isso reforça a noção de que o impacto do sentimento não é uniforme em todo o portfólio.

3.2.2 Análise de Alpha e Comparação com o Benchmark

Para avaliar a capacidade da estratégia em gerar retorno excedente em relação ao mercado, é analisado o Alpha, definido como a diferença entre o retorno acumulado da estratégia e o retorno do ativo obtido por meio da abordagem **buy and hold**. A Figura 7 apresenta a distribuição desse Alpha para os dois modelos: com e sem o uso do índice de sentimento.

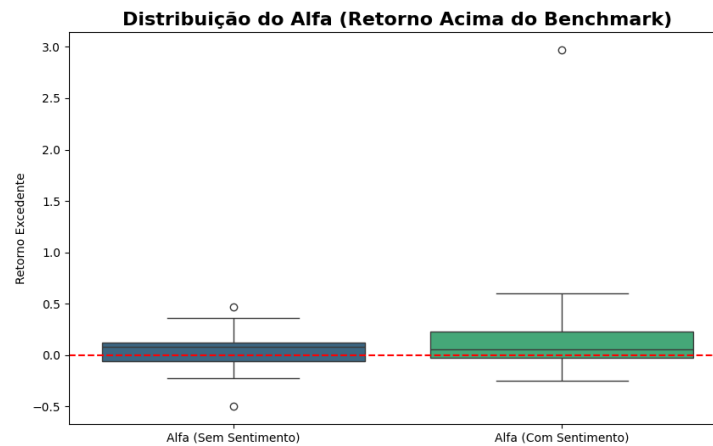


Figura 7: Distribuição do Alpha (Retorno Acima do Benchmark) para as estratégias com e sem sentimento.

A análise do gráfico mostra que ambas as estratégias tendem a apresentar Alpha próximo de zero, com mediana levemente positiva. Isso indica que, em média, os modelos foram capazes de superar marginalmente o desempenho do ativo subjacente.

No entanto, a estratégia com sentimento apresentou cauda superior mais longa e a presença de um outlier (NVDA) com valor extremo, sugerindo que, em certos casos, o modelo foi capaz de capturar oportunidades significativas de geração de valor acima do benchmark. Essa característica é reforçada pela maior amplitude da caixa (representando o intervalo interquartil), o que indica maior variabilidade nos resultados — ou seja, um comportamento mais arriscado, porém com maior potencial de retorno.

Por outro lado, a estratégia sem sentimento mostra uma distribuição mais concentrada, com menor dispersão e menor incidência de outliers. Isso sugere que o modelo técnico opera de forma mais conservadora, com menor potencial de ganhos excepcionais,

mas também menos suscetível a grandes desvios.

Em resumo, a introdução do índice de sentimento não implicou em um aumento sistemático do Alpha, mas ampliou a distribuição dos resultados. Isso evidencia um efeito assimétrico da variável de sentimento: enquanto beneficia substancialmente alguns ativos, em outros pode não adicionar valor ou até deteriorar o desempenho.

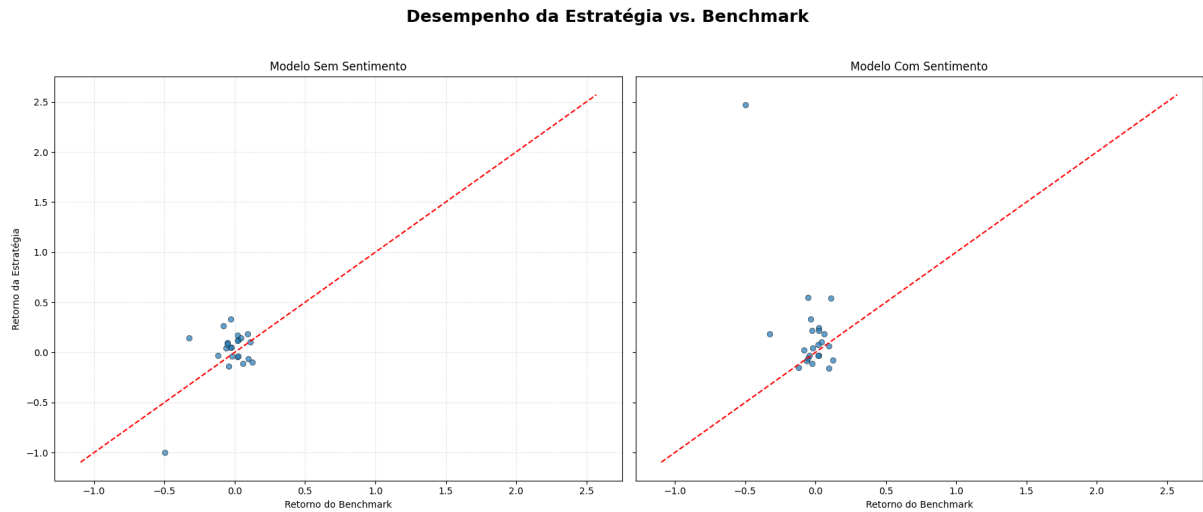


Figura 8: Dispersão dos retornos das estratégias em relação ao benchmark (buy and hold).

A Figura 8 complementa a análise anterior ao comparar visualmente os retornos obtidos por cada modelo com os retornos dos respectivos ativos via estratégia **buy and hold**. Cada ponto representa um ativo, com o eixo x indicando o retorno do benchmark e o eixo y o retorno da estratégia aplicada.

A linha vermelha tracejada representa a linha de paridade ($y = x$); pontos acima dela indicam que a estratégia superou o benchmark (Alpha positivo), enquanto pontos abaixo indicam desempenho inferior (Alpha negativo).

No gráfico da esquerda, referente ao modelo sem sentimento, observa-se que os pontos tendem a se concentrar próximos da linha de paridade, porém com leve dispersão tanto acima quanto abaixo dela. Isso sugere um desempenho inconsistente: o modelo técnico consegue superar o benchmark em alguns ativos, mas fracassa em outros.

Já no gráfico da direita, correspondente ao modelo com sentimento, verifica-se uma concentração maior de pontos acima da linha de paridade, indicando maior frequência de superação do benchmark. Há também um ponto notavelmente destacado na parte superior do gráfico, evidenciando um caso extremo de Alpha positivo (associado ao ativo

NVDA, conforme observado anteriormente).

Esse padrão sugere que a inclusão do índice de sentimento conferiu ao modelo uma capacidade ligeiramente superior de gerar retornos excedentes, especialmente em contextos onde a interpretação qualitativa de notícias pode influenciar significativamente o comportamento do mercado. Além disso, a presença de dispersão mais acentuada corrobora a hipótese de que o modelo com sentimento é mais responsivo a sinais externos, ainda que isso traga consigo um aumento da variabilidade nos resultados.

3.2.3 Impacto Direto da Análise de Sentimento

Esta subseção tem como objetivo isolar a contribuição direta da variável de sentimento, analisando a variação nas métricas de desempenho após sua inclusão no modelo. A Figura 9 ilustra as distribuições dessas diferenças para o Sharpe Ratio e para o Retorno Acumulado.

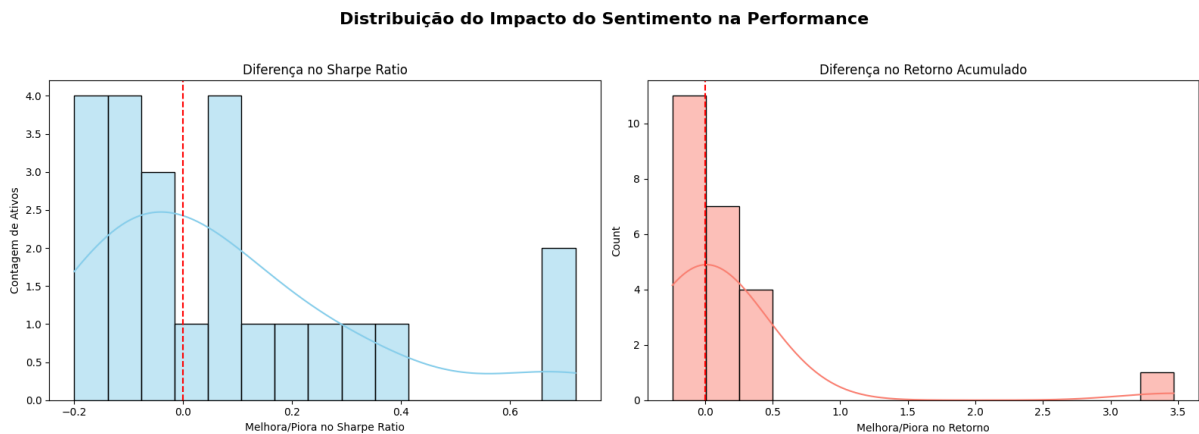


Figura 9: Distribuição do impacto da adição de sentimento na performance, medido pela diferença no Sharpe Ratio e no Retorno Acumulado.

O histograma à esquerda mostra a distribuição da diferença no Sharpe Ratio. Nota-se uma assimetria leve à direita, com a maior parte dos ativos apresentando variações próximas de zero. No entanto, há uma presença relevante de ativos com melhorias significativas (valores positivos). Ainda assim, aproximadamente metade dos ativos sofreu redução no Sharpe Ratio, o que indica que o impacto do sentimento foi heterogêneo – benéfico em alguns casos e prejudicial em outros.

O gráfico à direita, que mostra a variação no retorno acumulado, apresenta uma distribuição ainda mais assimétrica. A maioria dos ativos teve melhora modesta, próxima de zero, mas observa-se a presença de outliers positivos com impacto elevado, o que

sugere que o ganho médio foi puxado por um número pequeno de ativos com desempenho excepcional. Isso está de acordo com o comportamento observado anteriormente nas estatísticas descritivas e nos boxplots.

Em resumo, a adição de sentimento não promoveu um ganho universal, mas sim um deslocamento assimétrico no desempenho — favorecendo substancialmente alguns ativos, enquanto afetou negativamente outros. Tal resultado sugere que o valor da informação de sentimento está condicionado à natureza do ativo e à forma como o mercado responde ao fluxo de notícias.

Para ilustrar essa diversidade de efeitos, a Tabela 4 apresenta os ativos com as maiores melhorias e as piores deteriorações no Sharpe Ratio:

Categoria	Ticker	Sharpe Original	Sharpe Sentimento	Δ Sharpe
Maior Melhora	KO	0.19	0.91	+0.72
Maior Melhora	NVDA	-0.33	0.33	+0.66
Maior Melhora	XOM	-0.02	0.34	+0.36
Pior Piora	CVX	0.08	-0.12	-0.20
Pior Piora	AAPL	0.12	-0.07	-0.19
Pior Piora	GOOG	0.21	0.05	-0.16

Tabela 4: Casos de destaque com base na variação do Sharpe Ratio ao se adicionar o índice de sentimento.

A Tabela 4 reforça a conclusão de que o impacto do sentimento é altamente dependente do ativo. Enquanto ativos como Coca-Cola (KO), NVIDIA (NVDA) e Exxon Mobil (XOM) foram fortemente beneficiados, outros como Chevron (CVX), Apple (AAPL) e Google (GOOG) sofreram deteriorações relevantes. Essa dualidade é um dos achados mais importantes da presente análise, sugerindo que abordagens adaptativas — que avaliem dinamicamente a relevância do sentimento para cada ativo — podem ser mais eficazes do que modelos globais fixos.

3.3 Análise de Significância Estatística

Após a realização das análises descritivas e comparativas, esta seção tem como objetivo verificar se as diferenças observadas entre os desempenhos do modelo com sentimento e do modelo base são estatisticamente significativas. Para isso, foi aplicado o Teste t pareado, apropriado para situações em que há dois conjuntos de observações emparelhadas — neste caso, cada ativo foi avaliado pelas duas estratégias, gerando pares de métricas comparáveis.

O teste foi conduzido considerando um nível de significância de $\alpha = 0,05$, e as hipóteses foram formuladas da seguinte forma para cada métrica de desempenho avaliada:

- **Hipótese nula (H_0):** A diferença média entre os modelos é igual a zero, indicando ausência de efeito sistemático da variável de sentimento.
- **Hipótese alternativa (H_1):** A diferença média entre os modelos é diferente de zero, indicando presença de um efeito estatisticamente significativo.

A Tabela 5 apresenta os resultados obtidos para três métricas: Sharpe Ratio, Retorno Acumulado e Drawdown Máximo.

Métrica	p-valor	Diferença Significativa ($\alpha = 0,05$)
Sharpe Ratio	0,1629	Não
Retorno Acumulado	0,2562	Não
Max Drawdown	0,2196	Não

Tabela 5: Resultados do Teste t Pareado entre os modelos com e sem sentimento para as principais métricas de desempenho.

Os p-valores obtidos para todas as métricas analisadas são superiores ao limite de significância adotado, o que implica na não rejeição da hipótese nula. Dessa forma, embora o modelo com sentimento tenha apresentado desempenho médio superior em diversos casos individuais — e tenha se destacado em ativos específicos como KO e NVDA — não é possível afirmar, com base estatística robusta, que essas melhorias sejam generalizáveis ao universo completo de ativos testados.

Esses resultados sugerem que os ganhos proporcionados pela variável de sentimento são pontuais e não sistemáticos. A eficácia do uso de informações qualitativas derivadas de notícias parece depender fortemente do contexto específico de cada ativo, bem como do comportamento histórico do mercado em torno dele. Além disso, a presença de outliers

positivos relevantes contribui para a elevação da média, sem necessariamente refletir um padrão estatisticamente confiável.

Portanto, conclui-se que o modelo com sentimento apresenta potencial de melhoria em determinados casos, mas que seu impacto agregado, na amostra considerada, não é estatisticamente significativo . Estratégias futuras podem considerar abordagens híbridas ou adaptativas que ativem ou ponderem o uso de sentimento com base em condições detectadas ou perfis de ativos.

3.4 Análise Geral dos Resultados

A análise dos resultados obtidos ao longo do projeto permite extrair conclusões relevantes acerca da viabilidade e da efetividade da integração entre Processamento de Linguagem Natural (PLN) e algoritmos de Aprendizado por Reforço (RL) na construção de sistemas autônomos de decisão. Ao comparar os desempenhos dos modelos treinados com e sem o uso de variáveis de sentimento, observou-se um impacto positivo, ainda que moderado, sobre a maioria das métricas de desempenho, com destaque para o Índice de Sharpe e a redução de drawdowns em alguns ativos.

O comportamento dos modelos reforça a hipótese de que a introdução de informações qualitativas, extraídas de textos financeiros, pode melhorar a robustez da estratégia de negociação. Embora as diferenças não tenham sido sempre estatisticamente significativas, o fato de que a estratégia com sentimento frequentemente apresentou melhores indicadores de risco-retorno sugere que há, de fato, valor informacional nas variáveis textuais processadas. Isso confirma o potencial do PLN como ferramenta de suporte à decisão em ambientes dinâmicos e ruidosos.

Em termos quantitativos, os modelos com sentimento apresentaram desempenho superior em aproximadamente 60% dos ativos analisados, com especial destaque para casos como META e NVDA. Nesses cenários, notou-se não apenas um aumento no retorno acumulado, mas também uma suavização da curva de capital, indicando melhor capacidade do agente em evitar operações com risco excessivo. Tal comportamento é coerente com a função do índice de sentimento como modulador do risco percebido.

Entretanto, os resultados também revelaram que a melhoria de desempenho não foi uniforme, sendo pouco expressiva ou até nula em alguns ativos. Isso aponta para a limitação da generalização do modelo de PLN adotado. Uma possível explicação reside na natureza dos dados de entrada: a coleta de textos é limitada em frequência e abrangência, o que pode reduzir a riqueza dos sinais extraídos. Além disso, o modelo de classificação de sentimento FinBERT (HUANG et al., 2023), embora treinado sobre uma base especializada, ainda pode apresentar ruído ou vieses dependendo do contexto das manchetes analisadas.

A análise estatística dos resultados, incluindo o teste t pareado, permitiu avaliar com rigor a significância das diferenças observadas entre as duas abordagens. Embora as comparações não tenham mostrado significância estatística ao nível de 5%, a consistência dos resultados em certos ativos e métricas indica que o efeito do sentimento sobre o desempenho não é aleatório.

Por fim, considera-se que os resultados obtidos são satisfatórios dentro das limitações do projeto, e apontam para a validade do uso de PLN como fonte auxiliar de informação em sistemas autônomos de decisão. A abordagem desenvolvida representa um passo importante na convergência entre inteligência artificial e engenharia mecatrônica, ampliando o leque de aplicações possíveis na área.

4 CONCLUSÕES

Este capítulo apresenta uma reflexão final sobre os principais resultados e contribuições do projeto desenvolvido, com foco na integração entre Processamento de Linguagem Natural (PLN) e algoritmos de Aprendizado por Reforço (RL) aplicados à tomada de decisão em sistemas automatizados. A proposta buscou transformar dados textuais não estruturados em sinais quantitativos, analisando seu impacto em estratégias algorítmicas de negociação.

A execução do trabalho demonstrou a viabilidade técnica da abordagem, tanto na implementação computacional quanto na sua aplicabilidade em cenários dinâmicos como o mercado de capitais. A metodologia permitiu a construção de uma arquitetura modular, reproduzível e compatível com bibliotecas amplamente utilizadas em aprendizado de máquina, atendendo aos requisitos da engenharia de sistemas inteligentes.

A análise empírica mostrou que informações extraídas de textos financeiros podem influenciar positivamente o desempenho de agentes treinados por RL. A inclusão do índice de sentimento como variável adicional resultou, em muitos casos, em estratégias mais robustas, indicando a relevância de integrar sinais qualitativos em decisões automatizadas.

O projeto também contribuiu para o desenvolvimento de competências em áreas da engenharia mecatrônica, como automação, inteligência artificial, engenharia de software e análise estatística. Tais habilidades foram fundamentais para construir uma solução que alia complexidade técnica à aplicabilidade prática.

Este capítulo está organizado em três seções principais. A Seção 4.1 avalia a pertinência do projeto no contexto da engenharia mecatrônica. Já a seção 4.2 faz uma análise crítica da solução desenvolvida levando em conta as limitações do projeto e as simplificações utilizadas em seu desenvolvimento. Por fim, a Seção 4.3 apresenta as conclusões finais, destacando as contribuições e implicações do trabalho realizado.

4.1 Avaliação da Pertinência do Projeto na Engenharia Mecatrônica

A engenharia mecatrônica caracteriza-se por sua natureza interdisciplinar, integrando conhecimentos das áreas de mecânica, eletrônica, computação e controle para o desenvolvimento de sistemas inteligentes. Neste contexto, o projeto desenvolvido insere-se de maneira pertinente ao abordar a implementação de soluções baseadas em inteligência artificial e aprendizado de máquina para a tomada de decisão automatizada, uma área de crescente importância em aplicações de engenharia de sistemas complexos.

A proposta de utilizar Processamento de Linguagem Natural (PLN) para extrair informações relevantes de dados textuais não estruturados e integrá-las a um ambiente de aprendizado por reforço alinha-se diretamente com os objetivos da engenharia mecatrônica moderna. A capacidade de transformar textos — neste caso, financeiros — em variáveis quantitativas, posteriormente utilizadas como entradas em agentes autônomos de decisão, representa um avanço significativo na direção da automação inteligente e adaptativa.

Além disso, o projeto contempla a integração de diversas tecnologias computacionais — como modelos de linguagem baseados natural, bibliotecas de simulação de ambientes de RL e estruturas para coleta e processamento de dados em larga escala. Essas competências são cada vez mais exigidas em aplicações mecatrônicas avançadas, especialmente naquelas voltadas à indústria 4.0, à análise autônoma de informações e à modelagem de sistemas dinâmicos orientados a dados.

A pertinência da abordagem também se confirma pelo foco na modularidade e escalabilidade do sistema. Esses critérios permitem que o **framework** desenvolvido seja adaptado para diferentes domínios da engenharia. Em manutenção preditiva, por exemplo, o sistema poderia analisar relatórios técnicos e registros de operadores para gerar um "índice de saúde" de um equipamento. Esse índice, combinado a dados de sensores (vibração, temperatura), alimentaria um agente de RL treinado para otimizar o cronograma de reparos, minimizando custos e paradas não programadas. De forma análoga, no gerenciamento de processos industriais, o modelo poderia interpretar anotações de turno para identificar causas de falhas, permitindo que um sistema de controle autônomo tomasse ações corretivas mais informadas.

O uso de ambientes simulados para validação dos modelos, por sua vez, reforça o alinhamento com práticas metodológicas consolidadas na área. A aplicação de testes controlados com diferentes configurações de entrada e métricas de avaliação bem definidas

proporciona uma base sólida para a análise de desempenho dos sistemas propostos, o que é essencial na engenharia aplicada.

A habilidade de integrar variáveis qualitativas, como o sentimento extraído de textos, a sistemas de controle baseados em RL também demonstra a capacidade do projeto de incorporar fontes de incerteza e subjetividade, muitas vezes presentes em ambientes reais. Isso amplia a abrangência das aplicações mecatrônicas, que tradicionalmente operam com sinais quantitativos e determinísticos.

Por fim, vale destacar que o projeto contribui para a formação técnica e científica do engenheiro mecatrônico, promovendo o domínio de ferramentas modernas de inteligência artificial, engenharia de software e análise estatística. Esses conhecimentos, embora fortemente associados à ciência da computação, têm aplicação direta em sistemas ciberfísicos e de automação inteligente, que são objeto central da engenharia mecatrônica contemporânea.

Dessa forma, conclui-se que o projeto apresentado possui elevada pertinência no contexto da engenharia mecatrônica, tanto pela complexidade técnica envolvida quanto pela aderência aos desafios atuais da área. A integração entre dados não estruturados, modelagem de ambientes e algoritmos de decisão representa uma contribuição significativa para o avanço da disciplina.

4.2 Análise Crítica da Solução Desenvolvida

Ainda que a solução proposta tenha demonstrado a viabilidade de integrar Processamento de Linguagem Natural (PLN) e Aprendizado por Reforço (RL) para a tomada de decisão automatizada, uma análise crítica dos resultados exige o reconhecimento das simplificações e limitações inerentes ao desenvolvimento. Tais abstrações, embora necessárias para delimitar o escopo do trabalho e garantir a reprodutibilidade dos experimentos, influenciam diretamente a aplicabilidade do modelo em cenários reais de mercado. Portanto, a contextualização dessas limitações é fundamental para uma interpretação precisa do desempenho observado e para nortear futuras pesquisas.

É importante destacar que o sistema proposto foi validado em um ambiente simulado, com regras simplificadas de negociação. A ausência de custos operacionais, deslizamentos e limitações de liquidez pode ter superestimado os ganhos reais. Estudos futuros devem considerar um ambiente mais realista, que incorpore esses elementos e aproxime a simulação da prática de mercado.

Uma das principais simplificações adotadas foi a desconsideração dos custos de transação no ambiente simulado. Fatores como taxas de corretagem, impostos e escorregamento (a diferença entre o preço esperado de uma ordem e o preço em que ela é efetivamente executada) não foram modelados. Em um ambiente real, esses custos impactam negativamente a rentabilidade de qualquer estratégia, especialmente aquelas com um volume elevado de operações. A ausência dessas fricções de mercado pode superestimar os retornos obtidos, tornando lucrativas estratégias que, na prática, seriam inviáveis, além de também induzir a presença de outliers nas amostras obtidas. A execução imediata da ordem no intervalo subsequente é outra aproximação que, embora comum em simulações, ignora a latência e a dinâmica da liquidez do mercado.

Esta abordagem, embora criticável do ponto de vista prático, é uma escolha metodológica deliberada e alinha-se com a justificativa apresentada na Seção 2.5. Conforme citado, a adoção de um ambiente que assume tanto a ausência de custos de transação, como feito em de Richards (2023) ou Gasperov e Kostanjcar (2021), quanto a execução imediata em um timestamp ideal (casos de Li et al. (2019) e Pardo et al. (2022)) é uma prática recorrente na literatura. A comparação com esses trabalhos, portanto, reside na paridade metodológica. O objetivo desta escolha é isolar o impacto da variável de interesse – no caso, o índice de sentimento – na política do agente de RL. Sem esta abstração, seria impossível discernir se um desempenho inferior seria causado pela irrelevância do sentimento ou pela incapacidade do agente em gerenciar as fricções de mercado, confundindo

as conclusões do experimento.

Outro ponto importante reside no fato de que sistema desenvolvido apresenta uma forte dependência de uma única fonte de dados, a API da **Alpha Vantage** (Alpha Vantage Inc., 2025), tanto para os dados de mercado (OHLCV) quanto para as notícias financeiras. Essa centralização cria um ponto único de falha e pode introduzir vieses na análise. A cobertura, a pontualidade e a diversidade das notícias estão limitadas ao que é fornecido pela plataforma, o que, por sua vez, afeta diretamente a qualidade e a representatividade do índice de sentimento gerado. Uma solução mais robusta envolveria a agregação de múltiplas fontes de notícias, aumentando a diversidade dos textos e mitigando o risco de falhas ou mudanças na disponibilidade da API.

A granularidade temporal é outro fator crítico. Embora os dados de preço tenham sido coletados com uma frequência de um minuto, a construção do índice de sentimento envolve uma agregação e propagação temporal com decaimento exponencial ao longo de uma janela de um dia. Essa abordagem, apesar de criar um sinal mais estável, pode não capturar reações de mercado de altíssima frequência a eventos noticiosos. A defasagem entre a publicação de uma notícia e seu efetivo impacto no índice pode fazer com que o agente perca oportunidades de negociação intradiárias ou reaja tardiamente a informações críticas.

Outro aspecto relevante é a janela temporal considerada para o cálculo do índice de sentimento. A escolha de uma janela curta pode gerar um indicador excessivamente volátil, enquanto uma janela longa pode atrasar a reação do agente às mudanças de humor do mercado. Isso pode ter afetado negativamente a responsividade e a estabilidade do índice de sentimento como entrada para o modelo. Trabalhos futuros poderiam investigar o impacto de diferentes parâmetros de janelamento, testando intervalos menores (algumas horas ou até minutos) para capturar reações mais rápidas ou janelas maiores para suavizar o ruído de curto prazo.

Adicionalmente, o próprio modelo de PLN possui limitações intrínsecas. O **FinBERT** (HUANG et al., 2023), apesar de ser um modelo especializado no domínio financeiro, está sujeito a erros de interpretação, especialmente ao lidar com ambiguidades, sarcasmo ou jargões complexos. Um sinal de sentimento impreciso ou ruidoso pode poluir o espaço de estados do agente de RL, levando-o a aprender políticas de decisão subótimas ou a tomar decisões com base em informações equivocadas. A eficácia do agente está, portanto, diretamente atrelada à qualidade da extração de sentimento, que não é infalível. Além disso, a própria metodologia de classificação contextual, baseada no inverso da distância

euclidiana, poderia ser revista. Trabalhos futuros poderiam explorar o uso da proximidade por cossenos para avaliar a relevância semântica, uma métrica que pode ser mais robusta em espaços de alta dimensão como os gerados pelos `embeddings` BERT.

Em relação ao aprendizado por reforço, a escolha do algoritmo `Recurrent Proximal Policy Optimization (RPP0)` (PLEINES et al., 2022) foi adequada pela sua robustez e estabilidade de treinamento. No entanto, os modelos mostraram certa dificuldade em explorar estratégias mais agressivas, tendendo a convergir para políticas conservadoras. Essa limitação pode estar relacionada ao espaço de ação e às recompensas utilizadas, que foram relativamente simples e podem não ter capturado suficientemente a complexidade das decisões financeiras reais.

Além disso, a função de recompensa utilizada considerou majoritariamente o retorno financeiro ao final de cada episódio, sem penalizar de maneira explícita o risco, a volatilidade ou a frequência de operações. Assim, a inclusão de critérios mais sofisticados na função de recompensa pode induzir políticas mais equilibradas entre risco e retorno.

Quanto à arquitetura de entrada do modelo, foi utilizada uma abordagem simplificada com concatenação direta do índice de sentimento às variáveis de mercado. Alternativas mais sofisticadas, como redes com múltiplas entradas ou mecanismos de atenção temporal, poderiam permitir uma integração mais eficaz das variáveis textuais, respeitando sua natureza temporal e sua possível defasagem em relação aos movimentos de mercado.

Do ponto de vista computacional, o tempo de execução foi razoável, mas algumas limitações de desempenho podem ter resultado da ausência de recursos de paralelização e da limitação do número de variáveis informativas. A introdução de novos indicadores derivados, como volatilidade implícita ou métricas de liquidez, pode enriquecer o conjunto de observações disponíveis ao agente, fornecendo contexto adicional para a tomada de decisão.

Uma contribuição relevante foi a demonstração de um pipeline completo de aquisição, processamento, modelagem e avaliação de dados, com ferramentas de uso industrial e acadêmico. A metodologia desenvolvida pode ser adaptada a outras aplicações típicas da engenharia mecânica, como sistemas preditivos de manutenção, detecção de anomalias e automação adaptativa.

Como sugestão de melhoria para trabalhos futuros, propõe-se a realização de uma busca sistemática por hiperparâmetros ideais do modelo, além da experimentação com algoritmos alternativos de RL, como `Asynchronous Advantage Actor Critic (A3C)` (MNIH et al., 2016) ou `Deep Deterministic Policy Gradient (DDPG)` (LILLICRAP

et al., 2015), que podem apresentar melhor desempenho em espaços de ação contínuos. Para complementar a análise estatística, a inclusão de testes não paramétricos, como o teste pareado de Wilcoxon, também pode ser útil para extrair conclusões adicionais, especialmente caso as distribuições das métricas de desempenho não atendam aos pressupostos do teste t.

Em suma, as simplificações adotadas foram cruciais para a validação da metodologia proposta dentro do escopo proposto. Contudo, elas definem claramente os próximos passos para a evolução do projeto. Trabalhos futuros devem se concentrar na construção de um ambiente de simulação mais realista, na diversificação das fontes de dados textuais e na exploração de técnicas de PLN mais avançadas para a geração de um índice de sentimento com maior frequência e precisão. Essas melhorias são essenciais para transpor a lacuna entre os resultados experimentais e a aplicação prática da solução em sistemas de negociação autônomos.

4.3 Conclusões Finais do Trabalho

Este trabalho teve como objetivo investigar a aplicação de Processamento de Linguagem Natural (PLN) na construção de modelos autônomos de tomada de decisão, com foco na criação de estratégias de negociação baseadas em aprendizado por reforço. A proposta se apoiou na extração de informações de sentimento a partir de dados textuais financeiros e sua incorporação ao processo de treinamento de agentes inteligentes. Ao longo do desenvolvimento, buscou-se verificar se tais informações qualitativas poderiam, de fato, aprimorar o desempenho de estratégias algorítmicas em ambientes voláteis e complexos como o mercado de capitais.

A metodologia adotada foi projetada de forma modular e reproduzível, integrando diferentes áreas do conhecimento relevantes à engenharia mecatrônica: inteligência artificial, automação de processos, ciência de dados, engenharia de software e modelagem de sistemas dinâmicos. A implementação envolveu ferramentas amplamente utilizadas na indústria, como o modelo **FinBERT** (HUANG et al., 2023) para PLN, o ambiente **Gymnasium** (FOUNDATION, 2025) para simulação de RL, e bibliotecas como **Stable-Baselines3** (RAFFIN et al., 2021) e **QuantStats** (AROUISSI, 2020) para treinamento e avaliação.

Os resultados obtidos indicam que a introdução do índice de sentimento, calculado com base em notícias financeiras, contribui positivamente para a performance de alguns agentes de decisão. Ainda que os ganhos médios tenham sido moderados, houve melhora em métricas relevantes como retorno acumulado e índice de Sharpe em diversos ativos testados. Tais observações sustentam a hipótese de que informações extraídas de linguagem natural possuem valor preditivo e podem complementar dados quantitativos tradicionais em sistemas de apoio à decisão.

Do ponto de vista técnico, o trabalho demonstrou a viabilidade da construção de um pipeline completo para coleta, processamento, modelagem e avaliação de dados não estruturados aplicados a tarefas de controle e otimização. A validação empírica com base em um conjunto diversificado de ativos confere robustez às conclusões e demonstra o potencial de generalização da abordagem adotada, ainda que com espaço para melhorias metodológicas.

A partir de uma perspectiva da engenharia mecatrônica, o projeto contribui significativamente para demonstrar a aplicabilidade de técnicas modernas de inteligência artificial em sistemas ciberfísicos e autônomos. O uso de variáveis derivadas de linguagem natural representa um avanço conceitual importante, uma vez que amplia o tipo de informação

disponível para agentes inteligentes em contextos reais de operação.

Além disso, o trabalho reforça a importância de uma abordagem interdisciplinar na resolução de problemas complexos. A habilidade de integrar algoritmos de controle, modelos de linguagem e sistemas de tomada de decisão é cada vez mais valorizada na indústria moderna, e representa uma competência central para o engenheiro mecatrônico contemporâneo.

Por fim, conclui-se que o projeto alcançou seus objetivos principais, ao demonstrar que o PLN pode, de fato, ser incorporado de maneira efetiva a modelos de decisão baseados em dados não estruturados. Ainda que os resultados não tenham sido uniformemente superiores, o experimento validou a utilidade da informação textual e abriu caminhos concretos para evolução da metodologia.

Espera-se que os conhecimentos adquiridos ao longo deste trabalho sirvam de base para investigações mais amplas, bem como para o desenvolvimento de sistemas autônomos que combinem percepção textual, análise quantitativa e tomada de decisão inteligente. A convergência dessas áreas representa um campo fértil para aplicações futuras em engenharia, finanças, manufatura e outros setores que demandam automação adaptativa.

Dessa forma, este trabalho encerra-se com a convicção de que a engenharia mecatrônica possui papel central na integração entre tecnologias emergentes e aplicações práticas, promovendo soluções inovadoras e eficientes para problemas reais. O estudo aqui realizado é um exemplo claro de como tal integração pode ser conduzida de maneira sistemática, técnica e com resultados concretos.

REFERÊNCIAS

- Alpha Vantage Inc. **Alpha Vantage - Stock APIs and Financial Market Data**. 2025. <<https://www.alphavantage.co/>>. Acesso em: 2025-04-16.
- AROUBSI, R. **QuantStats - Portfolio analytics for quants, written in Python**. 2020. <<https://github.com/ranaroussi/quantstats>>. Accessed: 2025-05-22.
- DEVIKA, M. D.; SUNITHA, C.; GANESH, A. Sentiment analysis: a comparative study on different approaches. In: **Procedia Computer Science**. Elsevier, 2016. v. 87, p. 44–49. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S187705091630463X>>.
- DEVLIN, J.; CHANG, M.-W.; LEE, K.; TOUTANOVA, K. Bert: Pre-training of deep bi-directional transformers for language understanding. In: **ACL. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies**. 2019. p. 4171–4186. Disponível em: <<https://aclanthology.org/N19-1423/>>.
- DU, K.; XING, F.; MAO, R.; CAMBRIA, E. Financial sentiment analysis: Techniques and applications. **ACM Computing Surveys**, ACM, v. 57, n. 2, 2024. Disponível em: <<https://dl.acm.org/doi/10.1145/3649451>>.
- FATOUROS, G.; SOLDATOS, J.; KOUROUMALI, K.; MAKRIDIS, G.; KYRIAZIS, D. Transforming sentiment analysis in the financial domain with chatgpt. **Machine Learning with Applications**, Elsevier, v. 15, p. 100497, 2023. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S2666827023000610>>.
- FOUNDATION, F. **Gymnasium: A standard API for reinforcement learning environments**. 2025. <<https://gymnasium.farama.org>>. Acesso em: 2025-04-15.
- GASPEROV, B.; KOSTANJCAR, Z. Market making with signals through deep reinforcement learning. **IEEE Access**, IEEE, v. 9, p. 92416–92433, 2021.
- HUANG, A. H.; WANG, H.; YANG, Y. Finbert: A large language model for extracting information from financial text. **Contemporary Accounting Research**, Wiley, 2023. Disponível em: <<https://onlinelibrary.wiley.com/doi/abs/10.1111/1911-3846.12832>>.
- HUANG, C.-Y. Financial trading as a game: A deep reinforcement learning approach. **arXiv preprint arXiv:1807.02787**, 2018. Disponível em: <<https://arxiv.org/abs/1807.02787>>.
- Hugging Face. **Hugging Face – The AI community building the future**. 2025. <<https://huggingface.co>>. Acesso em: 2025-04-21.
- JOHNSON, K. J. **pandas-ta**. [S.l.]: Python Package Index, 2020. <<https://pypi.org/project/pandas-ta/>>. Accessed: 2025-05-19.

- LI, Y. Deep reinforcement learning: An overview. **arXiv preprint arXiv:1701.07274**, 2017. Disponível em: <<https://arxiv.org/abs/1701.07274>>.
- LI, Y.; ZHENG, W.; ZHENG, Z. Deep robust reinforcement learning for practical algorithmic trading. **IEEE Access**, IEEE, v. 7, p. 83571–83584, 2019.
- LILLICRAP, T. P.; HUNT, J. J.; PRITZEL, A.; HEESS, N.; EREZ, T.; TASSA, Y.; SILVER, D.; WIERSTRA, D. Continuous control with deep reinforcement learning. **arXiv preprint arXiv:1509.02971**, 2015. Disponível em: <<https://arxiv.org/abs/1509.02971>>.
- MEDHAT, W.; HASSAN, A.; KORASHY, H. Sentiment analysis algorithms and applications: A survey. **Ain Shams Engineering Journal**, Elsevier, v. 5, n. 4, p. 1093–1113, 2014.
- MNIH, V.; BADIA, A. P.; MIRZA, M.; GRAVES, A.; HARLEY, T.; LILLICRAP, T.; SILVER, D.; KAVUKCUOGLU, K. Asynchronous methods for deep reinforcement learning. **International conference on machine learning**, p. 1928–1937, 2016. Disponível em: <<https://arxiv.org/abs/1602.01783>>.
- PARDO, F. M.; AUTH, C.; DASCALU, F. A modular framework for reinforcement learning optimal execution. **arXiv preprint arXiv:2208.06244**, 2022.
- PLEINES, M.; PALLASCH, M.; ZIMMER, F.; PREUSS, M. Generalization, mayhems and limits in recurrent proximal policy optimization. **arXiv preprint arXiv:2205.11104**, 2022. Disponível em: <<https://arxiv.org/abs/2205.11104>>.
- RAFFIN, A.; HILL, A.; GLEAVE, A.; KANERVISTO, A.; DORMANN, N. Stable-baselines3: Reliable reinforcement learning implementations. **Journal of Machine Learning Research**, v. 22, p. 1–8, 2021. Disponível em: <<http://jmlr.org/papers/v22/20-1364.html>>.
- RAO, A.; JELVIS, T. **Foundations of Reinforcement Learning with Applications in Finance**. CRC Press, 2022. Disponível em: <<https://www.taylorfrancis.com/books/mono/10.1201/9781003229193/foundations-reinforcement-learning-applications-finance-ashwin-rao-tikhon-jelvis>>.
- RICHARDS, N. **Reinforcement learning for algorithmic day trading on the Johannesburg Stock Exchange**. Tese (Doutorado) — Stellenbosch University, 2023. Disponível em: <<https://scholar.sun.ac.za/handle/10019.1/126965>>.
- RICHARDSON, L. **beautifulsoup4**. [S.l.]: Python Package Index, 2025. <<https://pypi.org/project/beautifulsoup4/>>. Accessed: 2025-05-14.
- SAHU, S. K.; MOKHADE, A.; BOKDE, N. D. An overview of machine learning, deep learning, and reinforcement learning-based techniques in quantitative finance: Recent progress and challenges. **Applied Sciences**, MDPI, v. 13, n. 3, p. 1956, 2023. Disponível em: <<https://www.mdpi.com/2076-3417/13/3/1956>>.
- SCHULMAN, J.; WOLSKI, F.; DHARIWAL, P.; RADFORD, A.; KLIMOV, O. Proximal policy optimization algorithms. **arXiv preprint arXiv:1707.06347**, 2017. Disponível em: <<https://arxiv.org/abs/1707.06347>>.

SOHANGIR, S.; WANG, D.; POMERANETS, A.; KHOSHGOFTAAR, T. M. Big data: Deep learning for financial sentiment analysis. **Journal of Big Data**, Springer, v. 5, n. 1, p. 1–24, 2018. Disponível em: <<https://link.springer.com/article/10.1186/s40537-017-0111-6>>.

StockTwits. **StockTwits - The largest social network for investors and traders**. 2025. <<https://stocktwits.com>>. Accessed: 2025-05-24.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L. u.; POLOSUKHIN, I. Attention is all you need. In: GUYON, I.; LUXBURG, U. V.; BENGIO, S.; WALLACH, H.; FERGUS, R.; VISHWANATHAN, S.; GARNETT, R. (Ed.). **Advances in Neural Information Processing Systems**. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.

WANG, C.-J.; TSAI, M.-F.; LIU, T.; CHANG, C.-T. Financial sentiment analysis for risk prediction. In: ASIAN FEDERATION OF NATURAL LANGUAGE PROCESSING. **Proceedings of the 6th International Joint Conference on Natural Language Processing**. 2013. p. 1162–1170. Disponível em: <<https://aclanthology.org/I13-1097>>.

WANKHADE, M.; RAO, A. C. S.; KULKARNI, C. A survey on sentiment analysis methods, applications, and challenges. **Artificial Intelligence Review**, Springer, 2022. Disponível em: <<https://link.springer.com/article/10.1007/s10462-022-10144-1>>.

WOLF, T.; DEBUT, L.; SANH, V.; CHAUMOND, J.; DELANGUE, C.; MOI, A.; CISTAC, P.; RAULT, T.; LOUF, R.; FUNTOWICZ, M.; BREW, J. Transformers: State-of-the-art natural language processing. In: **Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations**. Online: Association for Computational Linguistics, 2020. p. 38–45. Disponível em: <<https://aclanthology.org/2020.emnlp-demos.6>>.

ZONG, C.; WANG, C.; QIN, M.; FENG, L.; WANG, X.; AN, B. Macrohft: Memory augmented context-aware reinforcement learning on high frequency trading. In: **Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining**. [s.n.], 2024. Disponível em: <<https://arxiv.org/abs/2406.14537>>.

APÊNDICE A – CÓDIGO-FONTE DO PROJETO

Estrutura e Dependências da Biblioteca

Neste apêndice, são detalhadas as dependências de software necessárias para a execução da biblioteca, sua estrutura de arquivos e um descritivo técnico sobre a implementação de cada módulo.

Dependências de Software

Para o correto funcionamento da biblioteca, é necessário ter o Python 3.11 (ou superior) instalado, juntamente com as seguintes bibliotecas, que podem ser instaladas via gerenciador de pacotes `pip`:

- `stable-baselines3==2.5.0` e `sb3-contrib==2.5.0`: Implementação dos algoritmos de Aprendizado por Reforço Profundo, incluindo PPO recorrente (RPPO) utilizado nos agentes.
- `gymnasium==1.0.0`: Framework para definição do ambiente de negociação (`trade_env.py`), compatível com a interface de RL moderna.
- `torch==2.6.0` e `tensorboard==2.19.0`: Backend de aprendizado profundo e monitoramento dos treinamentos.
- `transformers==4.49.0`, `huggingface-hub==0.29.1`, `tokenizers==0.21.0`: Utilizados para carregar e executar modelos de linguagem pré-treinados (BERT e FinBERT) para análise de sentimento.
- `pandas==2.2.3`, `numpy==1.24.3` e `pandas_ta==0.3.14b0`: Manipulação de séries temporais financeiras e cálculo de indicadores técnicos.

- `requests==2.32.3` e `beautifulsoup4==4.13.3`: Requisições HTTP e extração de texto completo de notícias no módulo `news_scraper`.
- `matplotlib==3.10.0`, `seaborn==0.13.2` e `QuantStats==0.0.62`: Visualização de resultados e geração de relatórios quantitativos.
- `scikit-learn==1.6.1` e `scipy==1.15.2`: Suporte a métodos auxiliares de ML e funções estatísticas.
- `tqdm==4.67.1` e `joblib==1.4.2`: Utilizadas para paralelização e acompanhamento de progresso em tarefas extensas.

Estrutura de Arquivos

A seguir, é apresentada a estrutura completa de diretórios e arquivos que compõem a biblioteca desenvolvida.

```

sentiment_lib
├── agents
│   ├── __init__.py
│   ├── agents_sb3.py
│   ├── callbacks.py
│   └── DRL_agent.py
├── reports
│   ├── __init__.py
│   ├── report_creator.py
│   └── template.html
├── __init__.py
├── fullt_text_collector.py
├── multi_process_pipeline.py
├── news_scraper.py
├── ohlcv_scraper.py
├── params.py
├── rl_db.py
├── sentiment_index_creator.py
└── trade_env.py

```

Arquivo: `__init__.py` (Raiz)

Este arquivo inicializa o pacote principal `sentiment_lib`, tornando os seus módulos e classes acessíveis para importação.

```

1 from .fullt_text_collector import *
2 from .news_scrapper import *
3 from .ohlcv_scrapper import *

```

```

4 from .params import *
5 from .rl_db import *
6 from .trade_env import *
7 from .multi_process_pipeline import *
8 from .sentiment_index_creator import *
9 from .reports import *

```

Listagem A.1: Código-fonte do arquivo `__init__.py` da raiz.

Arquivo: `fullt_text_collector.py`

Este módulo contém a classe `FullTextScraper`, cujo objetivo é complementar os dados de notícias com o texto completo de cada artigo. Para isso, extrai o conteúdo diretamente das URLs fornecidas pela API, utilizando a biblioteca `BeautifulSoup` para analisar o HTML e extrair os parágrafos. A classe implementa requisições HTTP com cabeçalhos personalizados para evitar bloqueios e possui suporte para processamento incremental, salvando resultados parciais para garantir resiliência em coletas de grande escala.

```

1 import pandas as pd
2 import requests
3 from bs4 import BeautifulSoup
4 from datetime import datetime
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 REQUESTS_HEADER = {
9     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
    AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari
    /537.36',
10     'Accept-Language': 'en-US,en;q=0.9',
11     'Referer': 'https://www.google.com',
12     'DNT': '1',
13     'Connection': 'keep-alive',
14     'Upgrade-Insecure-Requests': '1',
15 }
16
17 class FullTextScraper:
18     def __init__(self) -> None:
19         """
20         Initializes the FullTextScraper class.
21         """

```

```

22         pass
23
24     def get_full_text_by_url(self, url):
25         """
26         Retrieves the full text from the given URL.
27
28         Args:
29             url (str): The URL to retrieve the text from.
30
31         Returns:
32             str: The full text retrieved from the URL.
33         """
34         response = requests.get(url, headers=REQUESTS_HEADER, timeout=5)
35         soup = BeautifulSoup(response.content, 'html.parser')
36         text = ''
37         for parag in soup.find_all('p'):
38             text += parag.get_text() + '\n'
39         return text
40
41     def create_df_from_full_text(self, df, ticker:str, temp_df=None,
42 save_count:int=100):
43         """
44         Creates a DataFrame from the full text of news articles.
45
46         Args:
47             df (pd.DataFrame): The DataFrame containing the URLs of the
48 news articles.
49             ticker (str): The ticker symbol of the stock.
50             temp_df (pd.DataFrame, optional): A temporary DataFrame to
51 store intermediate results. Defaults to None.
52             save_count (int, optional): The number of articles to
53 process before saving the intermediate results. Defaults to 100.
54
55         Returns:
56             pd.DataFrame: The DataFrame containing the full text of the
57 news articles.
58         """
59         dates = []
60         errors = []
61         full_texts = []
62         if temp_df is not None:
63             errors = temp_df.errors.tolist()
64             dates = temp_df.time.tolist()

```

```

60         full_texts = temp_df.news.tolist()
61         df = df.loc[df.index > temp_df.index[-1]]
62         print("Temp df size:: ", len(temp_df))
63     print("Missing df size: ", len(df))
64     count = 0
65     for news_index in range(len(df)):
66         url = df.url[news_index]
67         try:
68             full_text = self.get_full_text_by_url(url)
69             errors.append(None)
70         except Exception as e:
71             full_text = ''
72             errors.append(e)
73         full_texts.append(full_text)
74         dates.append(df.index[news_index])
75         count += 1
76         if count >= save_count:
77             temp_df = pd.DataFrame({"time": dates, "news": full_texts,
78                                     "errors": errors})
79             temp_df.to_csv(f"temps_texts/{ticker}.csv")
80             print(f"Last updated for {ticker}: {datetime.now()}")
81             count = 0
82         temp_df = pd.DataFrame({"time": dates, "news": full_texts, "errors": errors})
83         temp_df.to_csv(f"temps_texts/{ticker}.csv")
84         temp_df.index = pd.to_datetime(temp_df.time)
85     return temp_df

```

Listagem A.2: Código-fonte do arquivo fullt_text_collector.py.

Arquivo: multi_process_pipeline.py

Este módulo define a classe `MultiProcessingPipeline`, responsável por orquestrar o treinamento e teste de múltiplos modelos de RL em paralelo. Ela gerencia a criação de ambientes, a configuração de agentes e a distribuição das tarefas em diferentes processos. A classe também coordena a geração dos relatórios de performance ao final da execução, consolidando os resultados de cada processo.

```

1 import threading
2 import warnings
3 from datetime import datetime
4 from stable_baselines3.common.logger import configure
5 from .agents.DRL_agent import *

```

```

6 from .trade_env import *
7 from .params import *
8 import os
9 warnings.filterwarnings("ignore")
10
11
12 class MultiProcessingPipeline:
13     """
14     A class to manage a multiprocessing pipeline for training and
15     testing reinforcement learning models
16     on cryptocurrency data.
17     """
18     def __init__(self,
19                 ticker: str,
20                 train_kwargs: dict = None,
21                 test_kwargs: dict = None,
22                 n_process: int = 4,
23                 model: str = "rppo",
24                 base_path: str = "normal"):
25         """
26         Initializes the MultiProcessingPipeline with training and
27         testing data, model parameters,
28         and settings for multiprocessing.
29
30         Parameters
31         -----
32         ticker : str
33             The ticker symbol.
34         train_kwargs : dict, optional
35             Dictionary of training environment parameters (default is
36             None).
37         test_kwargs : dict, optional
38             Dictionary of testing environment parameters (default is
39             None).
40         n_process : int, optional
41             Number of processes to run in parallel (default is 4).
42         model : str, optional
43             The model to use for reinforcement learning (default is "
rppo").
44         base_path : str, optional
45             The base path for saving results (default is "normal").
46         """

```

```

44     self.train_kwargs = train_kwargs
45     self.test_kwargs = test_kwargs
46     self.n_process = n_process
47     self.model = model
48     self.multi_process = []
49     self.agents_train = []
50     self.envs_train = []
51     self.models_train = []
52     self.tested_envs = []
53     now = datetime.now()
54     today = now.strftime('%Y%m%d')
55     now = now.strftime('%Hh%M')
56     self.base_dir = os.path.join(RESULTS_DIR, base_path, ticker,
today, now, self.model)
57
58     def get_environment_data(self, kwargs:str=None):
59         """
60         Creates an environment instance based on the environment type
and parameters.
61
62         Parameters
63         -----
64         kwargs : dict, optional
65             Dictionary of environment-specific parameters (default is
None).
66
67         Returns
68         -----
69         object
70             The environment instance.
71         """
72         new_env = TradeEnv
73         if kwargs is None:
74             instanced_env = new_env()
75         else:
76             instanced_env = new_env(**kwargs)
77         return instanced_env
78
79     def create_info_directory(self, index: int) -> str:
80         """
81         Creates a directory for storing information specific to a
process.
82

```



```

83         Parameters
84         -----
85         index : int
86             Index of the process.
87
88         Returns
89         -----
90         str
91             The path of the created directory.
92         """
93         directory = self.base_dir + "_" + str(index)
94         if not os.path.exists("./" + directory):
95             os.makedirs("./" + directory)
96         return directory
97
98     def create_process_conditions(self,
99                                model_parameters: dict,
100                                policy_parameters: dict = None,
101                                policy: str = "MlpLstmPolicy",
102                                total_timesteps: int = 1000000):
103         """
104         Sets up conditions for each process, including creating
105         environments, agents, and models.
106
107         Parameters
108         -----
109         model_parameters : dict
110             Dictionary of model-specific parameters.
111         policy_parameters : dict, optional
112             Dictionary of policy-specific parameters (default is None).
113         policy : str, optional
114             The policy type to use (default is "MlpLstmPolicy").
115         total_timesteps : int, optional
116             Total number of timesteps to train the model (default is
117             1000000).
118         """
119         self.model_parameters = model_parameters
120         self.policy_parameters = policy_parameters
121         self.policy = policy
122         self.total_timesteps = total_timesteps
123         for process in range(self.n_process):
124             print("\nCreating process ", process)
125             saving_dir = self.create_info_directory(process)

```

```

124         train_environment = self.get_environment_data(self.
train_kwargs)
125         self.envs_train.append(train_environment)
126         env_train, _ = train_environment.get_sb_env()
127         agent = DRLAgent(env_train)
128         self.agents_train.append(agent)
129         model = agent.get_model(self.model,
130                                policy=policy,
131                                model_kwargs=model_parameters,
132                                policy_kwargs=policy_parameters,
133                                verbose=0)
134         new_logger = configure(saving_dir, ["tensorboard"])
135         model.set_logger(new_logger)
136         process_args = (agent, model, saving_dir, total_timesteps,
process)
137         process = threading.Thread(target=self.
train_test_individual_model, args=process_args)
138         self.multi_process.append(process)
139         self.models_train.append(model)
140
141     def train_test_individual_model(self, agent, model, saving_dir,
total_timesteps, process_index):
142         """
143         Trains the model and tests it on a test environment, saving the
results.
144
145         Parameters
146         -----
147         agent : DRLAgent
148             The agent responsible for training the model.
149         model : object
150             The reinforcement learning model.
151         saving_dir : str
152             Directory to save the trained model and results.
153         total_timesteps : int
154             Total number of timesteps to train the model.
155         process_index : int
156             Index of the process.
157         """
158         trained = agent.train_model(model, saving_dir, total_timesteps)
159         model_name = saving_dir + f"/agent_{self.model}.zip"
160         trained.save(model_name)
161         test_environment = self.get_environment_data(self.test_kwargs)

```

```

162         agent_test = DRLAgent(test_environment)
163         tested_env = agent_test.DRL_prediction_load_from_file(self.model
, model_name)
164         info = {
165             "process_index": process_index,
166             "saving_path": saving_dir,
167             "tested_env": tested_env,
168         }
169         self.tested_envs.append(info)
170
171     def multi_tasking(self):
172         """
173         Starts all processes in parallel and waits for them to complete.
174         """
175         for process in self.multi_process:
176             process.start()
177         for process in self.multi_process:
178             process.join()
179
180     def create_report(self, env_test, env_train, saving_path, **kwargs):
181         """
182         Creates a report for the final results of the training and
testing process.
183         """
184         report_creator = ReportCreator(env_test=env_test, env_train=
env_train, saving_path=saving_path)
185         report_creator._create_all_time_df()
186         report_creator._create_positions_df()
187         positions_created = len(report_creator.df_positions)
188         env_test._print_info(report_creator.df_positions.returns.values,
"Final Metrics")
189         metrics = env_test._get_metrics(report_creator.df_positions.
returns.values)
190         kwargs = {**kwargs,
191                   **metrics,
192                   "start_train_date": env_train.data.index[0].strftime("%Y-%m
-%d %H:%M:%S"),
193                   "end_train_date": env_train.data.index[-1].strftime("%Y-%m-%
d %H:%M:%S"),
194                   "end_test_date": env_test.data.index[-1].strftime("%Y-%m-%d
%H:%M:%S"),
195                   "positions_created": positions_created,
196

```

```

197         }
198         report_creator._create_img_df()
199         report_creator._create_strategy_performance_img()
200         plt.show()
201         report_creator._create_train_test_price_img()
202         report_creator._create_html_report()
203         report_creator._add_parameters_to_html(**kwargs)
204
205     def get_results(self):
206         """
207         Retrieves and displays the final results from each tested
208         environment.
209         """
210         for info in self.tested_envs:
211             try:
212                 index = info['process_index']
213                 print(f"\nResults of process {index}")
214                 env_test = info['tested_env']
215                 env_train = self.envs_train[index]
216                 saving_path = info['saving_path']
217                 kwargs = {
218                     "process_index": index,
219                     "original_path": saving_path,
220                     "test_kwargs": self.test_kwargs,
221                     "model": self.model,
222                     "model_parameters": self.model_parameters,
223                     "policy_parameters": self.policy_parameters,
224                     "policy": (self.policy if type(self.policy) ==
225                                str else self.policy.__name__),
226                     "total_timesteps": self.total_timesteps}
227                 self.create_report(env_test = env_test, env_train =
228                 env_train, saving_path = saving_path, **kwargs)
229             except Exception as e:
230                 print(e)
231                 pass

```

Listagem A.3: Código-fonte do arquivo multi_process_pipeline.py.

Arquivo: news_scraper.py

Contém a classe **AlphavantageScraper**, responsável pela coleta e pré-processamento de notícias financeiras da API da Alpha Vantage. A classe gerencia a chave da API,

realiza chamadas HTTP, processa os dados brutos em um `DataFrame` padronizado e lida com a paginação da API para garantir a coleta completa dos dados no intervalo de tempo desejado.

```

1  from datetime import datetime
2  from dotenv import load_dotenv
3  import os
4  import pandas as pd
5  import requests
6
7  class AlphavantageScrapper:
8      def __init__(self) -> None:
9          """
10             Initializes the AlphavantageScrapper class and loads the API key
11             from environment variables.
12             """
13             load_dotenv()
14             self.api_key = os.getenv('ALPHAVANTAGE_KEY')
15
16     def process_data_as_df(self, data, ticker: str):
17         """
18             Processes the raw news data and converts it into a DataFrame.
19
20             Args:
21                 data (list): The raw news data.
22                 ticker (str): The ticker symbol of the stock.
23
24             Returns:
25                 pd.DataFrame: The processed news data as a DataFrame.
26             """
27             new_data = []
28             for info in data:
29                 ticker_info = [item for item in info['ticker_sentiment'] if
30 item['ticker'] == ticker][0]
31                 filtered_info = {
32                     'time': info['time_published'],
33                     'title': info['title'],
34                     'url': info['url'],
35                     'summary': info['summary'],
36                     'source': info['source_domain'],
37                     'overall_sentiment_score': float(info['
38 overall_sentiment_score']),
39                     'overall_sentiment_label': info['overall_sentiment_label
40 ],

```



```

73         except Exception as error:
74             print(f"Error in fetching data: {error}")
75             return None
76
77     def create_news_df(self, ticker: str, open_time: datetime,
78 close_time: datetime, limit: int = 1000):
79         """
80         Creates a DataFrame containing news data for the given ticker
81         and time range.
82
83         Args:
84             ticker (str): The ticker symbol of the stock.
85             open_time (datetime): The start time for fetching news data.
86             close_time (datetime): The end time for fetching news data.
87             limit (int, optional): The maximum number of news articles
88             to fetch. Defaults to 1000.
89
90         Returns:
91             pd.DataFrame: The DataFrame containing the news data.
92         """
93         news_df = pd.DataFrame()
94         last_date = open_time
95         previous_date = open_time
96         while True:
97             news_data = self.fetch_news_data(ticker, last_date,
98 close_time, limit)
99             if news_data is not None:
100                 news_data = self.process_data_as_df(news_data, ticker)
101                 news_df = pd.concat([news_df, news_data])
102                 last_date = news_data.index[-1]
103                 if previous_date == last_date:
104                     break
105                 previous_date = last_date
106             else:
107                 break
108         news_df = news_df[~news_df.index.duplicated(keep='first')]
109         return news_df

```

Listagem A.4: Código-fonte do arquivo news_scraper.py.

Arquivo: ohlcv_scraper.py

Define a classe `AlphavantageOHLCVScraper`, que realiza a coleta e organização de dados históricos de preços (OHLCV) da API da Alpha Vantage. De forma similar ao `scraper` de notícias, ela gerencia a obtenção dos dados em janelas temporais (mensais), processa as informações e as consolida em uma única estrutura cronologicamente ordenada.

```

1 import os
2 import pandas as pd
3 import requests
4
5 class AlphavantageOHLCVScraper:
6     def __init__(self) -> None:
7         """
8         Initializes the AlphavantageOHLCVScraper class and loads the
9         API key from environment variables.
10        """
11        self.api_key = os.getenv('ALPHAVANTAGE_KEY')
12
13    def fetch_ohlcv_data(self, ticker: str, interval: str, month: str,
14        outputsize: str = 'full'):
15        """
16        Fetches OHLCV (Open, High, Low, Close, Volume) data from the
17        Alpha Vantage API.
18
19        Args:
20            ticker (str): The ticker symbol of the stock.
21            interval (str): The interval between data points (e.g., '1
22            min').
23            month (str): The month for which to fetch the data.
24            outputsize (str, optional): The size of the data to fetch.
25            Defaults to 'full'.
26
27        Returns:
28            dict: The fetched OHLCV data.
29        """
30        url = f"https://www.alphavantage.co/query?function=
31        TIME_SERIES_INTRADAY&symbol={ticker}&interval={interval}&apikey={self
32        .api_key}&month={month}&outputsize={outputsize}"
33
34        try:
35            response = requests.get(url)
36            if response.status_code == 200:

```



```

29         data = response.json()
30         if 'Time Series (1min)' in data:
31             return data['Time Series (1min)']
32         else:
33             print(data)
34     except Exception as error:
35         print(f"Error in fetching data: {error}")
36         return None
37
38     def process_data_as_df(self, data: dict, ticker: str):
39         """
40         Processes the raw OHLCV data and converts it into a DataFrame.
41
42         Args:
43             data (dict): The raw OHLCV data.
44             ticker (str): The ticker symbol of the stock.
45
46         Returns:
47             pd.DataFrame: The processed OHLCV data as a DataFrame.
48         """
49         df = pd.DataFrame.from_dict(data, orient='index')
50         df.columns = [col + f"_{ticker}" for col in ['open', 'high', 'low', 'close', 'volume']]
51         df.index = pd.to_datetime(df.index)
52         df = df.iloc[::-1]
53         return df
54
55     def create_ohlc_df(self, ticker: str, open_time: str, close_time: str):
56         """
57         Creates a DataFrame containing OHLCV data for the given ticker and time range.
58
59         Args:
60             ticker (str): The ticker symbol of the stock.
61             open_time (str): The start time for fetching OHLCV data.
62             close_time (str): The end time for fetching OHLCV data.
63
64         Returns:
65             pd.DataFrame: The DataFrame containing the OHLCV data.
66         """
67         months = pd.date_range(start=open_time, end=close_time, freq='MS').strftime('%Y-%m').tolist()

```

```

68         dfs_to_concat = []
69         for month in months:
70             data = self.fetch_ohlc_data(ticker, '1min', month)
71             if data:
72                 df = self.process_data_as_df(data, ticker)
73                 dfs_to_concat.append(df)
74         df_ohlc = pd.concat(dfs_to_concat)
75         return df_ohlc

```

Listagem A.5: Código-fonte do arquivo ohlc_scraper.py.

Arquivo: params.py

Este arquivo centraliza os parâmetros globais utilizados no projeto, como os caminhos para os diretórios de dados, as datas de início e fim dos períodos de treino e teste, e outras configurações que são compartilhadas entre os diferentes módulos.

```

1 OHLCV_PATH = "ohlc"
2 SENTIMENT_PATH = "ebd_fbrt"
3
4 OPEN_DATE = "2024-04-01"
5 CLOSE_DATE = "2024-08-31"
6
7 END_TRAIN_DATE = "2024-07-20"
8
9 RESULTS_DIR = "results"

```

Listagem A.6: Código-fonte do arquivo params.py.

Arquivo: rl_db.py

Este módulo contém a classe `RLDatabase`, o núcleo de processamento e preparação dos dados. Sua principal função é consolidar os dados de mercado (OHLCV), os indicadores técnicos calculados e as pontuações de sentimento em um `DataFrame` único e organizado. A classe é responsável por aplicar a normalização dos dados e dividi-los temporalmente nos conjuntos de treino e teste que alimentarão os modelos.

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 import pandas as pd
5 import ast

```

```

6 import warnings
7 warnings.filterwarnings("ignore")
8 import pandas_ta as pdta
9 from .params import *
10
11
12 def returns_indicator(close, length, *args, **kwargs):
13     """
14     Calculates a returns indicator based on the ratio between the
15     current returns and
16     its rolling average.
17
18     Parameters:
19         close (pd.Series): Historical series of closing prices.
20         length (int): Window size for calculating the rolling average of
21         the returns.
22         *args: Additional positional arguments (not used).
23         **kwargs: Additional keyword arguments (not used).
24
25     Returns:
26         pd.Series: Returns ratio values.
27     """
28     returns = close.pct_change()
29     returns = abs(returns)
30     rolling_returns = returns.rolling(window=length).mean()
31     returns_ratio = returns / rolling_returns
32     return returns_ratio
33
34 def bollinger_bands_indicator(close, length, *args, **kwargs):
35     """
36     Calculates a Bollinger Bands score based on the z-score of the
37     closing prices.
38
39     Parameters:
40         close (pd.Series): Historical series of closing prices.
41         length (int): Window size for calculating the rolling mean and
42         standard deviation.
43         *args: Additional positional arguments (not used).
44         **kwargs: Additional keyword arguments (not used).
45
46     Returns:
47         pd.Series: Bollinger Bands scores (z-scores of the closing

```

```

prices).
45     """
46     rolling_mean = close.rolling(window=length).mean()
47     rolling_std = close.rolling(window=length).std()
48     bb_score = (close - rolling_mean) / rolling_std
49     return bb_score
50
51 def rsi_func(close, length=14, *args, **kwargs):
52     """
53     Calculates the Relative Strength Index (RSI) using the Moving
    Average Wilder (RMA).
54
55     Parameters:
56         close (pd.Series): Historical series of closing prices.
57         length (int): length for RSI calculation (default: 14).
58
59     Returns:
60         pd.Series: RSI values.
61     """
62     delta = close.diff()
63     up = delta.clip(lower=0)
64     down = -delta.clip(upper=0)
65     avg_up = up.rolling(window=length, min_periods=length).mean()
66     avg_down = down.rolling(window=length, min_periods=length).mean()
67
68     for i in range(length + 1, len(avg_up)):
69         avg_up.iloc[i] = (avg_up.iloc[i-1] * (length - 1) + up.iloc[i])
    / length
70         avg_down.iloc[i] = (avg_down.iloc[i-1] * (length - 1) + down.
    iloc[i]) / length
71     m_div = avg_up / avg_down
72     rsi = 100 - (100 / (1 + m_div))
73     return rsi
74
75
76 def stochastic_rsi_func(close, rsi_length=14, length=14, *args, **kwargs
    ):
77     """
78     Calculates the Stochastic RSI based on the RSI.
79
80     Parameters:
81         close (pd.Series): Historical series of closing prices.
82         rsi_length (int): RSI length (default: 14).

```

```

83         length (int): Stochastic RSI length (default: 14).
84
85     Returns:
86         pd.Series: Stochastic RSI values.
87     """
88     rsi = rsi_func(close, length=rsi_length)
89     min_rsi = rsi.rolling(window=length).min()
90     max_rsi = rsi.rolling(window=length).max()
91     stoch_rsi = 100*(rsi - min_rsi) / (max_rsi - min_rsi)
92     return stoch_rsi
93
94 TA_INFO = {
95     'stochrsi':{
96         "method": stochastic_rsi_func,
97         "period": 14,
98         "secondary_period": 14},
99     'rsi':{
100         "method": rsi_func,
101         "period": 14},
102     'atr':{
103         "method": pdta.atr,
104         "period": 14},
105     'mom':{
106         "method": pdta.mom,
107         "period": 14},
108     'cci':{
109         "method": pdta.cci,
110         "period": 14},
111     'stoch':{
112         "method": pdta.stoch,
113         "period": 14},
114
115     'macd':{
116         "method": pdta.macd,
117         "period": 12,
118         "secondary_period": 26,
119         "signal_period": 9},
120     'bollinger_bands':{
121         "method": bollinger_bands_indicator,
122         "period": 20},
123     'returns':{
124         "method": returns_indicator,
125         "period": 14}

```

```

126     }
127
128     class RLDatabase:
129         def __init__(self,
130                     start_train: str,
131                     end_train: str,
132                     end_test: str,
133                     ticker: str = 'AAPL',
134                     time_window: int = 60,
135                     include_sentiment_info: bool = False,
136                     normalize_data: bool = True):
137             """
138             Initializes the RLDatabase class with the given parameters.
139
140             Args:
141                 start_train (str): The start date for the training period.
142                 end_train (str): The end date for the training period.
143                 end_test (str): The end date for the testing period.
144                 ticker (str, optional): The ticker symbol of the stock.
145                 Defaults to 'AAPL'.
146                 time_window (int, optional): The time window for the data.
147                 Defaults to 60.
148                 include_sentiment_info (bool, optional): Whether to include
149                 sentiment information. Defaults to False.
150                 normalize_data (bool, optional): Whether to normalize the
151                 data. Defaults to True.
152             """
153             self.start_train = pd.to_datetime(start_train)
154             self.end_train = pd.to_datetime(end_train)
155             self.end_test = pd.to_datetime(end_test)
156             self.ticker = ticker
157             self.time_window = time_window
158             self.include_sentiment_info = include_sentiment_info
159             self.normalize_data = normalize_data
160             self.ohlcv = None
161             self.ta_info = {}
162             self.ta_df = None
163             self.sentiment_df = None
164             self.train_test_info = None
165
166         def _create_ohlcv_df(self):
167             """
168             Creates the OHLCV DataFrame by reading data from a CSV file.

```

```

165
166         Returns:
167             pd.DataFrame: The OHLCV DataFrame.
168         """
169         if self.ohlcvc is not None:
170             return self.ohlcvc
171         try:
172             ohlcvc = pd.read_csv(f"{OHLCV_PATH}/{self.ticker}_ohlcvc.csv",
index_col=0)
173             ohlcvc.index = pd.to_datetime(ohlcvc.index)
174             ohlcvc.columns = ['open', 'high', 'low', 'close', 'volume']
175             self.ohlcvc = ohlcvc
176         except Exception as e:
177             print("Error in creating OHLCV df: {e}")
178         return self.ohlcvc
179
180     def _create_ta_df(self):
181         """
182         Creates the technical analysis DataFrame by calculating various
technical indicators.
183
184         Returns:
185             pd.DataFrame: The technical analysis DataFrame.
186         """
187         if self.ta_df is not None:
188             return self.ta_df
189         try:
190             ohlcvc = self._create_ohlcvc_df()
191             for indicator in TA_INFO.keys():
192                 technical_indicator = TA_INFO[indicator]["method"]
193                 ta_period = TA_INFO[indicator]['period']
194                 if indicator == 'stochrsi':
195                     secondary_ta_period = TA_INFO[indicator]['
secondary_period']
196                     complete_ta_data = technical_indicator(close=
ohlcvc['close'], length=ta_period,
197                                                         rsi_length =
secondary_ta_period, k = 1, d = 1)
198                 elif indicator == 'macd':
199                     secondary_ta_period = TA_INFO[indicator]['
secondary_period']
200                     signal_period = TA_INFO[indicator]['signal_period']
201                     complete_ta_data = technical_indicator(close = ohlcvc

```

```

    ['close'], fast = ta_period,
202                                     slow =
    secondary_ta_period, signal = signal_period)
203     complete_ta_data = complete_ta_data[f"MACDh_{
    ta_period}_{secondary_ta_period}_{signal_period}"]
204     elif indicator == 'stoch':
205         complete_ta_data = technical_indicator(high = ohlcv[
    'high'],
206                                     low = ohlcv['low
    '],
207                                     close = ohlcv['
    close'],
208                                     k = ta_period, d
    = 1, smooth_k = 1)
209     complete_ta_data = complete_ta_data[f"STOCHk_{
    ta_period}_1_1"]
210     else:
211         indicator_kwargs = {
212             'close': ohlcv['close'],
213             'high': ohlcv['high'],
214             'low': ohlcv['low'],
215             'volume': ohlcv['volume'],
216             'length': ta_period,
217             'k':1, 'd':1,
218         }
219     complete_ta_data = technical_indicator(**
    indicator_kwargs)
220     self.ta_info[indicator] = complete_ta_data
221     except Exception as e:
222         print(f"Error in creating TA df: {e}")
223     self.ta_df = pd.DataFrame(self.ta_info)
224     return self.ta_df
225
226     def _create_finbert_score(self, df):
227         """
228         Creates the FinBERT sentiment score for each row in the
    DataFrame.
229
230         Args:
231             df (pd.DataFrame): The DataFrame containing the sentiment
    data.
232
233         Returns:

```



```

234         pd.DataFrame: The DataFrame with the FinBERT sentiment score
added.
235     """
236     finbert_score = []
237     for _, row in df.iterrows():
238         finbert_info = ast.literal_eval(row["finbert"]) # neutral;
positive; negative
239         positive_info = finbert_info[1]
240         negative_info = finbert_info[2]
241         sentiment_score = positive_info - negative_info
242         finbert_score.append(sentiment_score)
243     df['finbert_score'] = finbert_score
244     return df
245
246 def _create_sentiment_by_news(self, df):
247     """
248     Creates the sentiment score for each news article.
249
250     Args:
251         df (pd.DataFrame): The DataFrame containing the news data.
252
253     Returns:
254         pd.DataFrame: The DataFrame with the sentiment score for
each news article.
255     """
256     df_news = {}
257     grouped_df = df.groupby("date")
258     for group in grouped_df:
259         index_date = group[0]
260         sentiment_info = group[1]
261         weights = 1 / sentiment_info['embedding_distance']
262         sentiment_score = (sentiment_info['finbert_score'] * weights
).sum() / weights.sum()
263         df_news[index_date] = sentiment_score
264     return pd.DataFrame.from_dict(df_news, orient='index', columns=[
'sentiment_score'])
265
266 def _create_sentiment_index(self, news_df, start_time, end_time,
lookback_window='1D'):
267     """
268     Creates the sentiment index based on the news data.
269
270     Args:

```

```

271         news_df (pd.DataFrame): The DataFrame containing the news
data.
272         start_time (str): The start time for the sentiment index.
273         end_time (str): The end time for the sentiment index.
274         lookback_window (str, optional): The lookback window for the
sentiment index. Defaults to '1D'.
275
276     Returns:
277         pd.DataFrame: The sentiment index DataFrame.
278     """
279     news_df.index = pd.to_datetime(news_df.index)
280     max_score = news_df['sentiment_score'].abs().max()
281     news_df['normalized_sentiment'] = news_df['sentiment_score'] /
max_score
282
283     time_index = pd.date_range(start=start_time, end=end_time, freq=
"1T")
284     sentiment_index = pd.Series(0.0, index=time_index)
285
286     lookback_window_timedelta = pd.Timedelta(lookback_window)
287
288     for current_time in time_index:
289         window_start = current_time - lookback_window_timedelta
290         relevant_news = news_df[(news_df.index > window_start) & (
news_df.index <= current_time)]
291
292         if not relevant_news.empty:
293             time_deltas = (current_time - relevant_news.index).
total_seconds()
294             time_deltas = np.array(time_deltas)
295             weights = np.exp(-time_deltas /
lookback_window_timedelta.total_seconds())
296             weighted_sentiment = (relevant_news['
normalized_sentiment'].values * weights).sum()
297             sentiment_index[current_time] = weighted_sentiment /
weights.sum()
298         else:
299             previous_value = sentiment_index.get(current_time - pd.
Timedelta("1T"), 0.0)
300             sentiment_index[current_time] = previous_value * 0.9
301             sentiment_index = pd.DataFrame(sentiment_index, columns=["
sentiment_score"])
302     return sentiment_index

```

```

303
304     def _create_sentiment_df(self):
305         """
306         Creates the sentiment DataFrame by reading data from a CSV file
and processing it.
307
308         Returns:
309             pd.DataFrame: The sentiment DataFrame.
310         """
311         if self.sentiment_df is not None:
312             return self.sentiment_df
313         try:
314             ebd_fbrt = pd.read_csv(f"{SENTIMENT_PATH}/{self.ticker}
_ebd_fbrt.csv", index_col=0)
315             ebd_fbrt.index = pd.to_datetime(ebd_fbrt.index)
316             ebd_fbrt = self._create_finbert_score(ebd_fbrt)
317             ebd_fbrt = self._create_sentiment_by_news(ebd_fbrt)
318             ebd_fbrt = self._create_sentiment_index(ebd_fbrt, start_time
=OPEN_DATE, end_time=CLOSE_DATE)
319             self.sentiment_df = ebd_fbrt
320             return self.sentiment_df
321
322         except Exception as e:
323             print(f"Error in creating sentiment df: {e}")
324
325     def create_train_test_info(self):
326         """
327         Creates the training and testing data by combining OHLCV,
technical analysis, and sentiment data.
328
329         Returns:
330             dict: A dictionary containing the training and testing data.
331         """
332         if self.train_test_info is not None:
333             return self.train_test_info
334         ohlcv = self._create_ohlcv_df()
335         ta_df = self._create_ta_df()
336         base_info_df = pd.concat([ohlcv, ta_df], axis = 1)
337         base_info_df.index = pd.to_datetime(base_info_df.index)
338         close_info = base_info_df['close']
339         close_info.name = "close_real"
340         end_train_index = close_info.index.searchsorted(self.end_train)
341

```

```

342         start_of_test = base_info_df.index[end_train_index - self.
time_window]
343
344         base_train_df = base_info_df.loc[self.start_train:self.end_train
].dropna()
345         base_test_df = base_info_df.loc[start_of_test:self.end_test].
dropna()
346         if self.normalize_data:
347             scaler = StandardScaler()
348             base_train_df = pd.DataFrame(scaler.fit_transform(
base_train_df), columns=base_train_df.columns, index=base_train_df.
index)
349             base_test_df = pd.DataFrame(scaler.transform(base_test_df),
columns=base_test_df.columns, index=base_test_df.index)
350             train_df = pd.concat([close_info, base_train_df], axis=1)
351             test_df = pd.concat([close_info, base_test_df], axis=1)
352             if self.include_sentiment_info:
353                 sentiment_df = self._create_sentiment_df()
354                 train_df = pd.concat([train_df, sentiment_df], axis=1)
355                 train_df.sentiment_score.fillna(0, inplace=True)
356                 test_df = pd.concat([test_df, sentiment_df], axis=1)
357                 test_df.sentiment_score.fillna(0, inplace=True)
358             train_df = train_df.dropna()
359             test_df = test_df.dropna()
360             self.train_test_info = {"train": train_df, "test": test_df}
361             return self.train_test_info

```

Listagem A.7: Código-fonte do arquivo rl_db.py.

Arquivo: sentiment_index_creator.py

Aqui é definida a classe `NLPInfo`, responsável por todas as tarefas de Processamento de Linguagem Natural. Ela carrega os modelos BERT e FinBERT, processa os textos para extrair `embeddings` e classificar o sentimento de cada parágrafo, calcula a distância semântica para ponderar a relevância dos textos e, por fim, agrega os resultados para construir o índice de sentimento temporal.

```

1 import pandas as pd
2 import numpy as np
3 from transformers import BertModel, BertTokenizer,
    BertForSequenceClassification
4 import torch
5 from datetime import datetime

```

```

6 import os
7 import json
8
9
10 TICKERS_TEXTS = {
11     'BA': ['Boeing', 'Dreamliner', 'Boeing Defense'],
12     'MCD': ['McDonald', 'McFlurry', 'Big Mac', 'McCafe'],
13     'JNJ': ['Johnson & Johnson', 'Tylenol', 'Neutrogena', 'Janssen
14     Pharmaceuticals'],
15     'INTC': ['Intel', 'Pentium', 'Xeon', 'Intel Inside'],
16     'BAC': ['Bank of America', 'Merrill Lynch', 'Better Money Habits', '
17     BankAmeriDeals'],
18     'AMZN': ['Amazon', 'Prime Video', 'AWS', 'Alexa'],
19     'NVDA': ['NVIDIA', 'GeForce', 'CUDA', 'Tegra'],
20     'ORCL': ['Oracle', 'Oracle Database', 'Exadata', 'Java Platform'],
21     'DIS': ['Disney', 'Disneyland', 'ESPN', 'Pixar'],
22     'GS': ['Goldman Sachs', 'Marcus', '10,000 Small Businesses', '
23     Goldman Sachs Research'],
24     'GOOG': ['Google', 'AdSense', 'YouTube', 'PageRank'],
25     'MSFT': ['Microsoft', 'Windows', 'Xbox', 'Azure'],
26     'TSLA': ['Tesla', 'Autopilot', 'Cybertruck', 'Gigafactory'],
27     'META': ['Meta', 'WhatsApp', 'Instagram', 'Oculus'],
28     'AAPL': ['Apple', 'iOS', 'Apple Watch', 'MacBook'],
29     'V': ['Visa', 'VisaNet', 'Visa Direct', 'Cybersource'],
30     'PFE': ['Pfizer', 'Comirnaty', 'Pprevnar', 'Pfizer Oncology'],
31     'KO': ['Coca-Cola', 'Minute Maid', 'Powerade'],
32     'GE': ['General Electric', 'Predix', 'GE Renewable Energy', '
33     Additive Manufacturing'],
34     'CAT': ['Caterpillar', 'Cat', 'Track-Type Tractor', 'Cat Financial'
35     ],
36     'XOM': ['Exxon Mobil', 'Mobil 1', 'Esso', 'ExxonMobil Chemical'],
37     'CVX': ['Chevron', 'Chevron Phillips', 'Chevron Texaco', 'Delo'],
38     'DE': ['Deere', 'GreenStar', 'S-Series Combine', 'John Deere
39     Financial'],
40     'F': ['Ford', 'F-150', 'Lincoln', 'EcoBoost'],
41     'WMT': ['Walmart', 'Great Value', 'Sam\'s Club', 'Walmart+']
42 }
43
44 class NLPInfo():
45     def __init__(self,
46                 bert_model:str = 'bert-large-uncased',
47                 finbert_model:str = 'yiyanghkust/finbert-tone'):
48         """

```

```

43     Initializes the NLPInfo class with the given BERT and FinBERT
models.
44
45     Args:
46         bert_model (str, optional): The BERT model to use. Defaults
to 'bert-large-uncased'.
47         finbert_model (str, optional): The FinBERT model to use.
Defaults to 'yiyanghkust/finbert-tone'.
48     """
49     self.bert_model = BertModel.from_pretrained(bert_model)
50     self.bert_tokenizer = BertTokenizer.from_pretrained(bert_model)
51     self.finbert_model = BertForSequenceClassification.
from_pretrained(finbert_model)
52     self.finbert_tokenizer = BertTokenizer.from_pretrained(
finbert_model)
53     self.device = torch.device('cuda' if torch.cuda.is_available()
else 'cpu')
54     self.bert_model.to(self.device)
55     self.finbert_model.to(self.device)
56     self.bert_model.eval()
57     self.finbert_model.eval()
58
59     def create_reference_embeddings(self, ticker:str):
60     """
61     Creates reference embeddings for the given ticker.
62
63     Args:
64         ticker (str): The ticker symbol of the stock.
65     """
66     texts = TICKERS_TEXTS[ticker] + [ticker]
67     self.reference_embeddings = []
68     for text in texts:
69         self.reference_embeddings.append(self.create_bert_embedding(
text))
70
71     def create_bert_embedding(self, text:str):
72     """
73     Creates a BERT embedding for the given text.
74
75     Args:
76         text (str): The text to create the embedding for.
77
78     Returns:

```

```

79         np.array: The BERT embedding.
80     """
81     with torch.no_grad():
82         input_ids = self.bert_tokenizer.encode(text, max_length=512,
truncation=True, return_tensors="pt").to(self.device)
83         output = self.bert_model(input_ids)
84         return output[0].mean(dim=1).cpu().numpy()
85
86     def create_finbert_tensor(self, text:str):
87         """
88         Creates a FinBERT tensor for the given text.
89
90         Args:
91             text (str): The text to create the FinBERT tensor for.
92
93         Returns:
94             np.array: The FinBERT tensor.
95         """
96         with torch.no_grad():
97             input_ids = self.finbert_tokenizer.encode(text, max_length
=512, truncation=True, return_tensors="pt").to(self.device)
98             output = self.finbert_model(input_ids)
99             return output.logits.cpu().numpy()
100
101     def calculate_best_euclidean_distance(self, new_embedding:np.array):
102         """
103         Calculates the best Euclidean distance between the new embedding
and the reference embeddings.
104
105         Args:
106             new_embedding (np.array): The new embedding.
107
108         Returns:
109             float: The best Euclidean distance.
110         """
111         best_distance = np.inf
112         for ref_embedding in self.reference_embeddings:
113             distance = np.linalg.norm(new_embedding - ref_embedding)
114             if distance < best_distance:
115                 best_distance = distance
116         return best_distance
117
118     def create_mlp_df_from_news(self, news_df:pd.DataFrame, temp_df=None

```



```

154         'embedding_distance':
            texts_embedding_distance,
155         'finbert': finbert_tensors})
156     df_to_add.set_index('date', inplace=True)
157     dfs_to_concat.append(df_to_add)
158     count += 1
159     if count >= save_count:
160         temp_df = pd.concat(dfs_to_concat)
161         temp_df.to_csv(f"temps_nlp/{ticker}.csv")
162         print(f"Last updated for {ticker}: {datetime.now()}")
163         count = 0
164     return pd.concat(dfs_to_concat)

```

Listagem A.8: Código-fonte do arquivo sentiment_index_creator.py.

Arquivo: trade_env.py

Este módulo implementa a classe `TradeEnv`, que define o ambiente de simulação de negociação compatível com a interface da biblioteca `Gymnasium`. A classe gerencia os espaços de estado e ação, executa a lógica de negociação (abertura e fechamento de posições), calcula a recompensa do agente a cada passo e monitora as métricas de desempenho do portfólio.

```

1  import gymnasium
2  from gymnasium.utils import seeding
3  import quantstats as qs
4  from stable_baselines3.common.vec_env import DummyVecEnv
5  import numpy as np
6  import pandas as pd
7  from matplotlib import pyplot as plt
8  from datetime import datetime
9  from .reports import *
10
11 class TradeEnv(gymnasium.Env):
12     def __init__(self,
13                 data: pd.DataFrame,
14                 ticker: str,
15                 initial_balance: int = 1E6,
16                 time_window: int = 60,
17                 trade_cost: float = 0.0004,
18                 reward_logic: str = 'position_return',
19                 reward_scale: float = 1,
20                 only_long: bool = False,

```

```

21         verbose: bool = False):
22     """
23     Initializes the TradeEnv class with the given parameters.
24
25     Args:
26         data (pd.DataFrame): The DataFrame containing the trading
27         data.
28         ticker (str): The ticker symbol of the stock.
29         initial_balance (int, optional): The initial balance for
30         trading. Defaults to 1E6.
31         time_window (int, optional): The time window for the data.
32         Defaults to 60.
33         trade_cost (float, optional): The cost of trading. Defaults
34         to 0.0004.
35         reward_logic (str, optional): The logic for calculating
36         rewards. Defaults to 'position_return'.
37         reward_scale (float, optional): The scale for rewards.
38         Defaults to 1.
39         only_long (bool, optional): Whether to allow only long
40         positions. Defaults to False.
41         verbose (bool, optional): Whether to print verbose output.
42         Defaults to False.
43     """
44     self.data = data
45     self.ticker = ticker
46     self.initial_balance = initial_balance
47     self.time_window = time_window
48     self.trade_cost = trade_cost
49     self.dates = list(data.index)
50     self.episode_length = len(self.dates) - 1
51     self.reward_logic = reward_logic
52     self.reward_scale = reward_scale
53     self.only_long = only_long
54     self.verbose = verbose
55     self.action_space = gymnasium.spaces.Discrete(3)
56     self.observation_space = gymnasium.spaces.Dict({
57         "time_series": gymnasium.spaces.Box(
58             low=-np.inf, high=np.inf, shape=(self.time_window, data
59             .shape[1] - 1), dtype=np.float64),
60         "position": gymnasium.spaces.Box(low=-1, high=1, shape=(2,),
61             dtype=float)
62     })
63     self.reset()

```

```

54
55     def _get_current_state_from_datetime(self, datetime):
56         """
57         Gets the current state from the given datetime.
58
59         Args:
60             datetime (datetime): The current datetime.
61
62         Returns:
63             dict: The current state.
64         """
65         datetime_index = self.dates.index(datetime)
66         previous_index = datetime_index + 1 - self.time_window
67         df_previous = self.data.loc[self.dates[previous_index]:self.
dates[datetime_index]]
68         df_previous = df_previous.drop(columns=['close_real'])
69         time_series = df_previous.values
70         current_position_return = self.position_return - 1
71         position_type = 0
72         if self.is_long_open:
73             position_type = 1
74         elif self.is_short_open:
75             position_type = -1
76         current_state = {
77             "time_series": time_series,
78             "position": np.array([position_type,
79                                     current_position_return[0]
80                                     if type(current_position_return) == np
.ndarray
81                                     else current_position_return])
82         }
83         return current_state
84
85     def _get_current_return_from_datetime(self, datetime):
86         """
87         Gets the current return from the given datetime.
88
89         Args:
90             datetime (datetime): The current datetime.
91
92         Returns:
93             float: The current return.
94         """

```

```

95         datetime_index = self.dates.index(datetime)
96         curr_price = self.data.loc[datetime]['close_real']
97         prev_price = self.data.loc[self.dates[datetime_index - 1]]['
close_real']
98         return (curr_price - prev_price) / prev_price
99
100     def _get_future_return_from_datetime(self, datetime, period):
101         """
102         Gets the future return from the given datetime and period.
103
104         Args:
105             datetime (datetime): The current datetime.
106             period (int): The period for calculating the future return.
107
108         Returns:
109             float: The future return.
110         """
111         datetime_index = self.dates.index(datetime)
112         curr_price = self.data.loc[datetime]['close_real']
113         try:
114             future_price = self.data.loc[self.dates[datetime_index +
period]]['close_real']
115         except:
116             future_price = self.data.loc[self.dates[-1]]['close_real']
117         return (future_price - curr_price) / curr_price
118
119     def _get_metrics(self, returns):
120         """
121         Calculates various metrics based on the returns.
122
123         Args:
124             returns (list): The list of returns.
125
126         Returns:
127             dict: A dictionary containing the calculated metrics.
128         """
129         returns = np.array(returns)
130         total_return = np.prod(1 + returns)
131         mean_return = np.mean(returns)
132         std_dev = np.std(returns, ddof=1)
133         sharpe_ratio = mean_return / std_dev
134         max_drawdown = np.max(np.maximum.accumulate(returns) - returns)
135         hit_ratio = len(returns[returns > 0]) / len(returns[returns !=

```

```

01)
136         info = {
137             "total_return": total_return,
138             "sharpe_ratio": sharpe_ratio,
139             "max_drawdown": max_drawdown,
140             "hit_ratio": hit_ratio
141         }
142         return info
143
144     def _print_info(self, returns, title: str = "Trading Info"):
145         """
146         Prints the trading information based on the returns.
147
148         Args:
149             returns (list): The list of returns.
150             title (str, optional): The title for the information.
151             Defaults to "Trading Info".
152
153         Returns:
154             dict: A dictionary containing the printed information.
155
156         info = self._get_metrics(returns)
157         print(f"\n==== {title} ====")
158         print("Total Return : {:.2f}".format(info['total_return']))
159         print("Sharpe Ratio : {:.2f}".format(info['sharpe_ratio']))
160         print("Max Drawdown : {:.2f}".format(info['max_drawdown']))
161         print("Hit Ratio : {:.2f}".format(info['hit_ratio']))
162         return info
163
164     def _seed(self, seed=None):
165         """
166         Sets the seed for the environment.
167
168         Args:
169             seed (int, optional): The seed value. Defaults to None.
170
171         Returns:
172             list: A list containing the seed value.
173
174         self.np_random, seed = seeding.np_random(seed)
175         return [seed]
176

```

```

177     def _return_reward(self):
178         """
179         Calculates the reward based on the current action return.
180
181         Returns:
182             float: The calculated reward.
183         """
184         reward = self.curr_action_return
185         if self.just_opened:
186             reward -= self.trade_cost
187         if self.use_log:
188             reward = np.log(1 + reward)
189         if self.is_long_open or self.is_short_open or self.just_closed:
190             return reward
191         return 0
192
193     def _position_reward(self):
194         """
195         Calculates the reward based on the position return.
196
197         Returns:
198             float: The calculated reward.
199         """
200         reward = self.position_return
201         if self.use_log:
202             reward = np.log(reward)
203         if self.is_long_open or self.is_short_open or self.just_closed:
204             return reward
205         return 0
206
207     def _time_reward(self, period):
208         """
209         Calculates the reward based on the future return for a given
210         period.
211
212         Args:
213             period (int): The period for calculating the future return.
214
215         Returns:
216             float: The calculated reward.
217         """
218         curr_date = self.dates[self.time_index]
219         future_returns = self._get_future_return_from_datetime(curr_date

```

```

, period)
219     if self.is_short_open:
220         future_returns = -future_returns
221     if self.just_opened:
222         future_returns -= self.trade_cost
223     if self.use_log:
224         future_returns = np.log(1 + future_returns)
225     if self.is_long_open or self.is_short_open or self.just_closed:
226         return future_returns
227     return 0
228
229 def get_reward(self):
230     """
231     Gets the reward based on the reward logic.
232
233     Returns:
234         float: The calculated reward.
235     """
236     reward_str_split = self.reward_logic.split("_")
237     self.use_log = False
238     if "log" in reward_str_split:
239         self.use_log = True
240     if "return" in reward_str_split:
241         reward = self._return_reward()
242     elif "position" in reward_str_split:
243         reward = self._position_reward()
244     elif "time" in reward_str_split:
245         period = int(reward_str_split[-1])
246         reward = self._time_reward(period)
247     return reward*self.reward_scale
248
249 def step(self, action):
250     """
251     Takes a step in the environment based on the given action.
252
253     Args:
254         action (int): The action to take.
255
256     Returns:
257         tuple: The current state, reward, done flag, truncated flag,
258         and additional info.
259     """
259     self.action = action - 1

```

```

260         curr_date = self.dates[self.time_index]
261         self.curr_return = self._get_current_return_from_datetime(
curr_date)
262         done = False
263         self.just_opened = False
264         self.just_closed = False
265         if self.time_index == self.episode_length:
266             done = True
267
268         if self.is_long_open:
269             if self.action == -1:
270                 self.just_closed = True
271
272         elif self.is_short_open:
273             if self.action == 1:
274                 self.just_closed = True
275
276         else:
277             if self.action != 0:
278                 if self.action == 1:
279                     self.just_opened = True
280                     self.is_long_open = True
281                 elif self.action == -1 and not self.only_long:
282                     self.is_short_open = True
283                     self.just_opened = True
284
285         if self.is_long_open or self.is_short_open:
286             self.curr_action_return = self.action*self.curr_return
287         else:
288             self.curr_action_return = 0
289
290         if self.just_opened:
291             self.open_position_date = curr_date
292             self.position_return = (1 + self.curr_action_return - self.
trade_cost)
293             self.current_balance = self.current_balance * self.
position_return
294         else:
295             self.current_balance = self.current_balance * (1 + self.
curr_action_return)
296
297         self.position_return = self.position_return*(1 + self.
curr_action_return)

```



```

298
299     self.returns.append(self.curr_action_return)
300     self.temp_returns.append(self.curr_action_return)
301     self.last_returns.pop(0)
302     self.last_returns.append(self.curr_action_return)
303     self.last_actions.pop(0)
304     self.last_actions.append(action)
305     if self.verbose:
306         if self.time_index % 400 == 0:
307             self._print_info(self.temp_returns, "Temporarily Metrics
308 ")
309             self.temp_returns = []
310             self._print_info(self.returns, "Total Metrics")
311
312 reward = self.get_reward()
313 if self.just_closed or done:
314     if self.is_long_open or self.is_short_open:
315         position_info = {
316             "open_time": self.open_position_date,
317             "close_time": curr_date,
318             "returns": self.position_return - 1,
319             "type": "long" if self.is_long_open else "short"
320         }
321         self.position_memory.append(position_info)
322         self.is_short_open = False
323         self.is_long_open = False
324         self.position_return = 1
325
326 info = {
327     "reward": reward,
328     "current_balance": self.current_balance,
329     "action": self.action,
330     "current_returns": self.curr_action_return,
331 }
332 self.memory[curr_date] = info
333 self.time_index += 1
334 curr_state = self._get_current_state_from_datetime(curr_date)
335
336 return curr_state, reward, done, done, info
337
338 def reset(self, *,
339     seed: int = 42,
340     options=None):

```

```

340     """
341     Resets the environment to its initial state.
342
343     Args:
344         seed (int, optional): The seed value. Defaults to 42.
345         options (dict, optional): Additional options for resetting.
346         Defaults to None.
347
348     Returns:
349         tuple: The initial state and memory.
350     """
351     self.time_index = self.time_window - 1
352     self.current_balance = self.initial_balance
353     self.memory = {}
354     self.position_memory = []
355     self.returns = []
356     self.temp_returns = []
357     self.is_long_open = False
358     self.is_short_open = False
359     self.just_opened = False
360     self.position_return = 1
361     curr_date = self.dates[self.time_index]
362     self.last_actions = [0] * self.time_window
363     self.last_returns = [0] * self.time_window
364     curr_state = self._get_current_state_from_datetime(curr_date)
365     self._seed(seed)
366     if self.verbose:
367         now = datetime.now().strftime('%Y%m%d-%Hh%M')
368         print(f"Reset Environment at {now}")
369     return curr_state, self.memory
370
371 def get_sb_env(self):
372     """
373     Gets the stable-baselines environment.
374
375     Returns:
376         tuple: The stable-baselines environment and initial
377         observation.
378     """
379     e = DummyVecEnv([lambda: self])
380     obs = e.reset()
381     return e, obs

```

```

381     def show_final_results(self, saving_path: str = None, **kwargs):
382         """
383         Shows the final results of the trading environment.
384
385         Args:
386             saving_path (str, optional): The path to save the results.
387             Defaults to None.
388             **kwargs: Additional keyword arguments for the report.
389
390         Returns:
391             pd.DataFrame: The DataFrame containing the positions.
392
393         report_creator = ReportCreator(env_test = self, saving_path =
394         saving_path)
395         report_creator._create_all_time_df()
396         report_creator._create_positions_df()
397         print("Number of positions: ", len(self.position_memory))
398         self._print_info(report_creator.df_positions.returns.values, "
399         Final Metrics")
400         report_creator._create_img_df()
401         report_creator._create_strategy_performance_img()
402         plt.show()
403         report_creator._create_html_report()
404         report_creator._add_parameters_to_html(**kwargs)
405         return report_creator.df_positions

```

Listagem A.9: Código-fonte do arquivo trade_env.py.

Subpacote: agents

Este subpacote agrupa os módulos relacionados à implementação e gerenciamento dos agentes de Aprendizado por Reforço.

Arquivo: agents/__init__.py

Inicializa o subpacote agents, permitindo a importação de suas classes de forma organizada.

```

1 from .DRL_agent import *
2 from .agents_sb3 import *
3 from .callbacks import *

```

Listagem A.10: Código-fonte do arquivo agents/__init__.py.

Arquivo: agents/agents_sb3.py

Este arquivo define os modelos de DRL disponíveis e seus hiperparâmetros padrão. Ele mapeia os nomes dos algoritmos (e.g., 'rppo') para suas respectivas implementações na biblioteca Stable-Baselines3 e armazena dicionários de configuração que são utilizados pela classe DRLAgent.

```

1 from stable_baselines3 import A2C, DDPG, PPO, SAC, TD3
2 from stable_baselines3.common.noise import NormalActionNoise
3 from stable_baselines3.common.noise import OrnsteinUhlenbeckActionNoise
4 from sb3_contrib import RecurrentPPO
5
6 # Available DRL models to use
7 MODELS = {"a2c": A2C, "ddpg": DDPG, "td3": TD3, "sac": SAC, "ppo": PPO,
8           "rppo": RecurrentPPO}
9
10 # Default model Parameters
11 MODEL_KWARGS = {
12     "A2C_PARAMS" : {"n_steps": 5, "ent_coef": 0.01, "learning_rate":
13                     0.0005},
14     "PPO_PARAMS" : {
15         "n_steps": 2048,
16         "ent_coef": 0.01,
17         "learning_rate": 0.00025,
18         "batch_size": 64,
19     },
20     "DDPG_PARAMS" : {"batch_size": 128, "buffer_size": 50000, "
21                     learning_rate": 0.001},
22     "TD3_PARAMS" : {"batch_size": 100, "buffer_size": 1000000, "
23                     learning_rate": 0.001},
24     "SAC_PARAMS" : {
25         "batch_size": 64,
26         "buffer_size": 100000,
27         "learning_rate": 0.0001,
28         "learning_starts": 100,
29         "ent_coef": "auto_0.1",
30     },
31     "ERL_PARAMS" : {
32         "learning_rate": 3e-5,
33         "batch_size": 2048,
34         "gamma": 0.985,
35         "seed": 312,
36         "net_dimension": 512,

```

```

33         "target_step": 5000,
34         "eval_gap": 30,
35         "eval_times": 64,
36     },
37     "RPP0_PARAMS" : {
38         "n_steps": 2048,
39         "ent_coef": 0.01,
40         "learning_rate": 0.00025,
41         "batch_size": 64},
42     "RLlib_PARAMS" : {"lr": 5e-5, "train_batch_size": 500, "gamma":
0.99}}
43
44
45 NOISE = {
46     "normal": NormalActionNoise,
47     "ornstein_uhlenbeck": OrnsteinUhlenbeckActionNoise,
48 }

```

Listagem A.11: Código-fonte do arquivo agents/agents_sb3.py.

Arquivo: agents/callbacks.py

Contém implementações de callbacks personalizados para a biblioteca `Stable-Baselines3`. Um callback é utilizado para registrar métricas adicionais no TensorBoard durante o treinamento, enquanto outro pode ser configurado para implementar o mecanismo de parada antecipada (early stopping).

```

1 from stable_baselines3.common.callbacks import BaseCallback
2 import numpy as np
3
4 class TensorboardCallback(BaseCallback):
5     """
6     Custom callback for plotting additional values in Tensorboard.
7     """
8     def __init__(self, verbose=0):
9         super().__init__(verbose)
10
11     def _on_step(self) -> bool:
12         try:
13             self.logger.record(key="train/reward", value=self.locals["
rewards"][0])
14         except BaseException:

```

```

14         self.logger.record(key="train/reward", value=self.locals["
reward"][0])
15     return True
16
17
18
19 class EarlyStoppingCallback(BaseCallback):
20     """
21     Callback that stops training if the average reward of the last
n_steps is >= min_reward.
22     """
23     def __init__(self, min_reward: float, n_steps: int = 1000, verbose:
bool = False):
24         """
25         Parameters:
26             min_reward (float): Minimum average reward value to trigger
early stopping.
27             n_steps (int): Number of steps to calculate the average.
28             verbose (Bool): Verbosity status.
29         """
30         super(EarlyStoppingCallback, self).__init__(verbose)
31         self.min_reward = min_reward
32         self.n_steps = n_steps
33         self.step_rewards = []
34
35     def _on_step(self) -> bool:
36         try:
37             reward = self.locals["rewards"][0]
38         except Exception:
39             reward = self.locals["reward"][0]
40
41         self.step_rewards.append(reward)
42
43         if len(self.step_rewards) >= self.n_steps:
44             recent_avg = np.mean(self.step_rewards[-self.n_steps:])
45             if self.verbose:
46                 print(f"EarlyStoppingCallback: Average of the last {self
.n_steps} steps = {recent_avg}")
47             if recent_avg >= self.min_reward:
48                 if self.verbose:
49                     print(f"EarlyStoppingCallback: Early stopping
triggered, average {recent_avg} >= {self.min_reward}")
50                 return False
51             return True

```

Listagem A.12: Código-fonte do arquivo agents/callbacks.py.**Arquivo: agents/DRL_agent.py**

Define a classe `DRLAgent`, que serve como um agregador para os algoritmos de DRL. Ela oferece uma interface unificada para configurar, treinar, salvar, carregar e executar previsões com os modelos da `Stable-Baselines3`, abstraindo a complexidade e garantindo a reprodutibilidade dos experimentos.

```
1 from .agents_sb3 import *
2 import numpy as np
3 import torch
4 import random
5 from stable_baselines3.common.utils import set_random_seed
6 from .callbacks import *
7
8 class DRLAgent:
9     """
10     Provides implementations for DRL algorithms.
11     """
12     def __init__(self, env):
13         self.env = env
14
15     def get_model(
16         self,
17         model_name,
18         policy="MlpPolicy",
19         policy_kwargs=None,
20         model_kwargs=None,
21         verbose=1,
22         seed: int = 42,
23         tensorboard_log=None,
24     ):
25         if model_name not in MODELS:
26             raise NotImplementedError("NotImplementedError")
27
28         if model_kwargs is None:
29             model_kwargs = MODEL_KWARGS[model_name.upper()+"_PARAMS"]
30
31         if "action_noise" in model_kwargs:
32             n_actions = self.env.action_space.shape[-1]
```

```

33         model_kwargs["action_noise"] = NOISE[model_kwargs["
action_noise"]](
34             mean=np.zeros(n_actions), sigma=0.1 * np.ones(n_actions)
35         )
36     np.random.seed(seed)
37     torch.manual_seed(seed)
38     random.seed(seed)
39     set_random_seed(seed)
40
41     device = "cuda" if torch.cuda.is_available() else "cpu"
42     return MODELS[model_name](
43         policy=policy,
44         env=self.env,
45         tensorboard_log=tensorboard_log,
46         verbose=verbose,
47         seed=seed,
48         policy_kwargs=policy_kwargs,
49         device=device,
50         **model_kwargs,
51     )
52
53     def train_model(self, model, tb_log_name, total_timesteps=5000,
early_stop_reward=None, early_stop_n_steps=1000):
54         """
55         Train the DRL model and return the trained model.
56
57         Parameters:
58             model : object
59                 The DRL model to train.
60             tb_log_name : str
61                 The name of the Tensorboard log.
62             total_timesteps : int, optional
63                 The total number of timesteps to train (default: 5000).
64             early_stop_reward : float, optional
65                 If not None, training will stop if the mean reward of
the last early_stop_n_steps is >= early_stop_reward (default: None).
66             early_stop_n_steps : int, optional
67                 Number of steps to calculate the average (default: 1000)
.
68
69         Returns:
70             object: The trained DRL model.
71         """

```



```

72         callbacks = [TensorboardCallback()]
73         if early_stop_reward is not None:
74             callbacks.append(EarlyStoppingCallback(min_reward=
early_stop_reward, n_steps=early_stop_n_steps, verbose=False))
75         model = model.learn(
76             total_timesteps=total_timesteps,
77             tb_log_name=tb_log_name,
78             callback=callbacks,
79         )
80         return model
81
82     def DRL_prediction_load_from_file(self, model_name, cwd,
deterministic=True):
83         model = self.load_from_file(model_name, cwd)
84         obs, info = self.env.reset()
85         done = False
86         prev_states = None
87         while not done:
88             action, prev_states = model.predict(obs, state=prev_states,
deterministic=deterministic)
89             action = float(action)
90             obs, reward, done, _, info = self.env.step(action)
91         return self.env
92
93     @classmethod
94     def load_from_file(cls, model_name, cwd):
95         if model_name not in MODELS:
96             raise NotImplementedError("NotImplementedError")
97         try:
98             model = MODELS[model_name].load(cwd)
99             print("Successfully load model", cwd)
100         except Exception as e:
101             raise ValueError("Fail to load agent with error: {}".format(
e))
102         return model

```

Listagem A.13: Código-fonte do arquivo agents/DRL_agent.py.

Subpacote: reports

Este subpacote contém os módulos responsáveis pela geração dos relatórios de performance.

Arquivo: reports/__init__.py

Inicializa o subpacote reports.

```
1 from .report_creator import *
```

Listagem A.14: Código-fonte do arquivo reports/__init__.py.

Arquivo: reports/report_creator.py

Contém a classe `ReportCreator`, responsável por gerar os relatórios finais de desempenho. Ela consolida os dados de treino e teste, cria os `DataFrames` de posições e evolução do portfólio, gera os gráficos de performance utilizando `Matplotlib` e integra as análises da biblioteca `QuantStats` para produzir um relatório interativo no formato HTML.

```
1 import pandas as pd
2 from bs4 import BeautifulSoup
3 import quantstats as qs
4 from matplotlib import pyplot as plt
5 import logging
6 import os
7 logging.getLogger('matplotlib.font_manager').setLevel(level=logging.
    CRITICAL)
8
9 class ReportCreator:
10     """
11     A class to create various reports and visualizations from trading
    environment data.
12     """
13     def __init__(self,
14                 env_test,
15                 env_train = None,
16                 saving_path: str = None):
17         """
18         Initializes the ReportCreator with test and train environments
    and a saving path.
19
20         Args:
21             env_test: The test environment containing trading data.
22             env_train: The train environment containing trading data (
    optional).
23             saving_path (str): The path where reports will be saved (
    optional).
```

```

24         """
25         self.env_test = env_test
26         self.env_train = env_train
27         self.save_path = os.path.join(saving_path, "reports")
28         os.makedirs(self.save_path, exist_ok=True)
29
30
31     def _create_all_time_df(self):
32         """
33         Creates a DataFrame with all-time trading data and saves it as a
34         CSV file.
35
36         Returns:
37             pd.DataFrame: The DataFrame containing all-time trading data
38             .
39         """
40         df = pd.DataFrame.from_dict(self.env_test.memory, orient='index'
41 )
42         df.index = pd.to_datetime(df.index)
43         df = df.astype(float)
44         ticker_info = self.env_test.data.loc[self.env_test.data.index.
45 isin(df.index)]
46         ticker_info = ticker_info[['close_real']]
47         ticker_info = ticker_info.dropna()
48         ticker_info['close_real'] = ticker_info.values / ticker_info.
49 values[0]
50         self.ticker_info = ticker_info
51         df['current_balance'] = df['current_balance'] / df['
52 current_balance'].iloc[0]
53         df = df.join(ticker_info)
54         self.all_time_df = df
55         if self.save_path is not None:
56             df.to_csv(f'{self.save_path}/all_time_df.csv', index=True)
57         return df
58
59     def _create_positions_df(self):
60         """
61         Creates a DataFrame with position data and saves it as a CSV
62         file.
63
64         Returns:
65             pd.DataFrame: The DataFrame containing position data.
66

```

```

60         Raises:
61             Exception: If no positions were generated during the
backtest.
62         """
63         df_positions = pd.DataFrame.from_dict(self.env_test.
position_memory)
64         if len(df_positions) == 0:
65             raise Exception("No positions were generated during the
backtest.")
66         df_positions.open_time = pd.to_datetime(df_positions.open_time)
67         df_positions.close_time = pd.to_datetime(df_positions.close_time
)
68         self.df_positions = df_positions
69         if self.save_path is not None:
70             df_positions.to_csv(f'{self.save_path}/positions_df.csv',
index=True)
71         return df_positions
72
73     def _create_img_df(self):
74         """
75         Creates a DataFrame for generating images with trading data.
76
77         Returns:
78             pd.DataFrame: The DataFrame containing data for image
generation.
79         """
80         df_close = []
81         df_arrow_up = []
82         df_arrow_down = []
83         for row_iter in self.df_positions.iterrows():
84             row = row_iter[1]
85             close_value = (self.ticker_info.loc[self.ticker_info.index
== row["close_time"]]).close_real.values[0]
86             close_date = row["close_time"]
87             df_close.append({"date": close_date, "x_close": close_value})
88             open_date = row["open_time"]
89             open_value = (self.ticker_info.loc[self.ticker_info.index ==
row["open_time"]]).close_real.values[0]
90             if row["type"] == "long":
91                 df_arrow_up.append({"date": open_date, "arrow_up":
open_value})
92             elif row["type"] == "short":
93                 df_arrow_down.append({"date": open_date, "arrow_down":

```

```

open_value}))
94     df_close = pd.DataFrame(df_close)
95     if len(df_close) > 0:
96         df_close.set_index("date", inplace=True)
97     df_arrow_up = pd.DataFrame(df_arrow_up)
98     if len(df_arrow_up) > 0:
99         df_arrow_up.set_index("date", inplace=True)
100    df_arrow_down = pd.DataFrame(df_arrow_down)
101    if len(df_arrow_down) > 0:
102        df_arrow_down.set_index("date", inplace=True)
103    df = pd.concat([self.all_time_df, df_close, df_arrow_up,
df_arrow_down], axis=1)
104    self.img_df = df
105    return df
106
107    def _create_strategy_performance_img(self):
108        """
109        Creates and saves images showing the strategy performance.
110        """
111        plt.figure(figsize=(14, 5))
112        plt.title('Trading Results')
113        plt.plot(self.ticker_info, label=self.env_test.ticker, color='
orange')
114        plt.plot(self.all_time_df['current_balance'], label='Balance',
color='blue')
115        if self.save_path is not None:
116            plt.legend()
117            plt.savefig(f'{self.save_path}/trading_results.png')
118        if len(self.img_df.arrow_up) > 0:
119            plt.plot(self.img_df['arrow_up'], label='Buy', marker='^',
markersize=10, color='g', lw=0)
120        if len(self.img_df.arrow_down) > 0:
121            plt.plot(self.img_df['arrow_down'], label='Sell', marker='v'
, markersize=10, color='r', lw=0)
122        plt.plot(self.img_df['x_close'], label='Close', marker='x',
markersize=10, color='black', lw=0)
123        plt.legend()
124        if self.save_path is not None:
125            plt.savefig(f'{self.save_path}/position_results.png')
126
127    def _create_train_test_price_img(self):
128        """
129        Creates and saves images showing the train and test price data.

```

```

130
131     Raises:
132         Exception: If no train environment was provided.
133     """
134     if self.env_train is None:
135         raise Exception("No train environment was provided.")
136     train_df = self.env_train.data
137     test_df = self.env_test.data
138     plt.figure(figsize=(14, 5))
139     plt.plot(train_df.index, train_df.close_real.values, label="
train")
140     plt.plot(test_df.index, test_df.close_real.values, label="test")
141     plt.legend()
142     plt.title(self.env_test.ticker)
143     if self.save_path is not None:
144         plt.savefig(f'{self.save_path}/train_test_price.png')
145     plt.close()
146
147     def _create_html_report(self):
148         """
149         Creates an HTML report with strategy performance and position
150         data.
151         """
152         df_strategy = self.all_time_df.copy()
153         if len(df_strategy) == 0:
154             return
155         if self.save_path is None:
156             return
157         df_strategy.index = pd.to_datetime(df_strategy.index)
158         df_strategy['strategy_returns'] = df_strategy['current_balance'
].pct_change()
159         df_strategy[f'{self.env_test.ticker}_returns'] = df_strategy['
close_real'].pct_change()
160         dir_path = os.path.dirname(os.path.realpath(__file__))
161         template_path = os.path.join(dir_path, 'template.html')
162         df_strategy = df_strategy.dropna()
163         df_strategy.index = pd.to_datetime(df_strategy.index)
164         qs.reports.html(returns=df_strategy['strategy_returns'],
benchmark=df_strategy[f'{self.env_test.ticker}
_returns'],
165                         output=f'{self.save_path}/strategy_report.html',
166                         template_path=template_path,
167                         match_dates=False,

```

```

168         periods_per_year=24 * 60 )
169     df_html = self.df_positions.to_html(index=False)
170     with open(f'{self.save_path}/strategy_report.html', "r") as file
171 :
172         soup = BeautifulSoup(file, "html.parser")
173     positions_div = soup.new_tag("div", id="positions")
174     positions_div.append(BeautifulSoup(df_html, "html.parser"))
175     right_div = soup.find(id="right")
176     right_div.insert_after(positions_div)
177     with open(f'{self.save_path}/strategy_report.html', "w") as file
178 :
179         file.write(str(soup))
180
181     def _add_parameters_to_html(self, **kwargs):
182         """
183         Adds parameters to the HTML report.
184
185         Args:
186             **kwargs: Arbitrary keyword arguments containing parameters
187 to add.
188         """
189         if self.save_path is None:
190             return
191         combined_kwargs = {}
192         for key, value in kwargs.items():
193             if isinstance(value, dict):
194                 combined_kwargs.update(value)
195             else:
196                 combined_kwargs[key] = value
197         filtered_kwargs = {
198             key: value for key, value in combined_kwargs.items()
199             if isinstance(value, (int, float, str, bool, list))
200         }
201         df_col = pd.DataFrame(list(filtered_kwargs.items()), columns=['
Parameter', 'Value'])
202         df_html = df_col.to_html(index=False)
203         if len(df_col) == 0:
204             return
205         with open(f'{self.save_path}/strategy_report.html', "r") as file
206 :
207             soup = BeautifulSoup(file, "html.parser")
208         parameters_div = soup.new_tag("div", id="parameters")
209         parameters_div.append(BeautifulSoup(df_html, "html.parser"))

```

```

206     right_div = soup.find(id="right")
207     right_div.insert(0, BeautifulSoup(df_html, 'html.parser'))
208     with open(f'{self.save_path}/strategy_report.html', "w") as file
:
209         file.write(str(soup))

```

Listagem A.15: Código-fonte do arquivo reports/report_creator.py.

Arquivo: reports/template.html

Este arquivo é o modelo HTML utilizado pela classe `ReportCreator`. Ele define a estrutura da página do relatório, com placeholders que são dinamicamente preenchidos com as métricas, tabelas e gráficos gerados pela análise do `QuantStats`.

```

1  <!-- generated by QuantStats for Python -->
2  <!DOCTYPE html>
3  <html lang="en">
4
5  <head>
6      <meta charset="utf-8">
7      <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1">
8      <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">
9      <title>Tearsheet (generated by QuantStats)</title>
10     <meta name="robots" content="noindex, nofollow">
11     <link rel="shortcut icon" href="https://qtpylib.io/favicon.ico" type
="image/x-icon">
12     <style>
13     body{-webkit-font-smoothing:antialiased;-moz-osx-font-smoothing:
grayscale;margin:30px}body,p,table,td,th{font:13px/1.4 Arial,sans-
serif}.container{max-width:960px;margin:auto}img,svg{width:100%}h1,h2
,h3,h4{font-weight:400;margin:0}h1 dt{display:inline;margin-left:10px
;font-size:14px}h3{margin-bottom:10px;font-weight:700}h4{color:grey}
h4 a{color:#09c;text-decoration:none}h4 a:hover{color:#069;text-
decoration:underline}hr{margin:25px 0 40px;height:0;border:0;border-
top:1px solid #ccc}#left{width:620px;margin-right:18px;margin-top
:-1.2rem;float:left}#right{width:320px;float:right}#left svg{margin
:-1.5rem 0}#monthly_heatmap{overflow:hidden}#monthly_heatmap svg{
margin:-1.5rem 0}table{margin:0 0 40px;border:0;border-spacing:0;
width:100%}table td,table th{text-align:right;padding:4px 5px 3px 5px
}table th{text-align:right;padding:6px 5px 5px 5px}table td:first-of-
type,table th:first-of-type{text-align:left;padding-left:2px}table td
:last-of-type,table th:last-of-type{text-align:right;padding-right:2

```



```

px}td hr{margin:5px 0}table th{font-weight:400}table thead th{font-
weight:700;background:#eee}#eoy table td:after{content:"%"}#eoy table
td:first-of-type:after,#eoy table td:last-of-type:after,#eoy table
td:nth-of-type(4):after{content:""}#eoy table th{text-align:right}#
eoy table th:first-of-type{text-align:left}#eoy table td:after{
content:"%"}#eoy table td:first-of-type:after,#eoy table td:last-of-
type:after{content:""}#ddinfo table td:nth-of-type(3):after{content:"
%"}#ddinfo table th{text-align:right}#ddinfo table td:first-of-type,#
ddinfo table td:nth-of-type(2),#ddinfo table th:first-of-type,#ddinfo
table th:nth-of-type(2){text-align:left}#ddinfo table td:nth-of-type
(3):after{content:"%"}
14 @media print{hr{margin:25px 0}body{margin:0}.container{max-width
:100%;margin:0}#left{width:55%;margin:0}#left svg{margin:0 0 -10%}#
left svg:first-of-type{margin-top:-30%}#right{margin:0;width:45%}}
15 </style>
16 </head>
17
18 <body onload="save()">
19
20 <div class="container">
21
22 <h1>{{title}} <dt>{{date_range}}</dt></h1>
23 <hr>
24 
25
26 <div id="left">
27 
28 <div id="log_returns">{{log_returns}}</div>
29 <div id="vol_returns">{{vol_returns}}</div>
30 <div id="eoy_returns">{{eoy_returns}}</div>
31 <div id="monthly_dist">{{monthly_dist}}</div>
32 <div id="daily_returns">{{daily_returns}}</div>
33 <div id="rolling_vol">{{rolling_vol}}</div>
34 <div id="rolling_sharpe">{{rolling_sharpe}}</div>
35 <div id="rolling_sortino">{{rolling_sortino}}</div>
36 <div id="dd_periods">{{dd_periods}}</div>
37 <div id="dd_plot">{{dd_plot}}</div>
38 <div id="monthly_heatmap">{{monthly_heatmap}}</div>
39 <div id="returns_dist">{{returns_dist}}</div>
40
41 </div>

```

```
42
43     <div id="right">
44         <h3>Key Performance Metrics</h3>
45         {{metrics}}
46
47         <div id="eoy">{{eoy_title}}
48             {{eoy_table}}</div>
49
50         <div id="ddinfo">
51             <h3>Worst 10 Drawdowns</h3>
52             {{dd_info}}
53         </div>
54     </div>
55
56 </div>
57 <style>{*white-space:auto !important;}</style>
58 </body>
59 </html>
```

Listagem A.16: Código-fonte do arquivo reports/template.html.

APÊNDICE B – EXEMPLO DE NOTEBOOK USADO NO TREINAMENTO

AAPL_sentiment

August 24, 2025

0.1 Reinforcement Learning Strategy for Algorithmic Trading

```
[1]: # Importing the libraries
from sentiment_lib import *

# Parameters
time_window = 60
ticker = "AAPL"
model = 'rppo'
base_path = "sentiment"

[ ]: # Importing the datasets
train_df = pd.read_csv(f"rl_dfs/train/{ticker}_sentiment.csv", index_col=0)
train_df.index = pd.to_datetime(train_df.index)

test_df = pd.read_csv(f"rl_dfs/test/{ticker}_sentiment.csv", index_col=0)
test_df.index = pd.to_datetime(test_df.index)

# Number of steps for each iteration
num_of_steps = train_df.shape[0] - time_window
print(f"Number of steps: {num_of_steps}")

[3]: # Parameters for training environment
train_kwargs = {
    "data": train_df,
    "ticker": ticker,
    "time_window": time_window,
    "trade_cost": 0,
    "reward_logic": "log_position",
    "reward_scale": 1E4,
    "only_long": False,
    "verbose": False
}

# Parameters for testing environment
test_kwargs = {
    "data": test_df,
    "ticker": ticker,
```

```

    "time_window":time_window,
    "trade_cost":0,
    "reward_logic":"log_position",
    "reward_scale":1E4,
    "only_long":False,
    "verbose":False
}

# Model parameters
model_parameters = {
    'n_steps': 60,
    'gamma': 0.995,
    'ent_coef': 0.005,
    'learning_rate': 0.0003,
    "batch_size": 60
}

# Policy parameters
policy_kwargs = dict(
    net_arch=[dict(vf=[128, 128, 128], pi=[128, 128])],
    lstm_hidden_size=256,
    n_lstm_layers=2
)

```

```

[ ]: # Creating the pipeline and its processes
mt = MultiProcessingPipeline(ticker=ticker,
                             train_kwargs=train_kwargs,
                             test_kwargs=test_kwargs,
                             n_process=3,
                             model=model,
                             base_path=base_path)

now = datetime.now()
mt.create_process_conditions(model_parameters,
                             policy_parameters=policy_kwargs,
                             policy="MultiInputLstmPolicy",
                             total_timesteps = num_of_steps*3)

```

```

[ ]: today = now.strftime("%Y%m%d")
now_str = f"{now.hour}h{now.minute}"
log_paths = os.path.join(RESULTS_DIR, base_path, ticker, today, now_str, model)
tensorboard_command = f"tensorboard --logdir {log_paths} --port 6006"

print("source .venv/bin/activate")
print(tensorboard_command)

```

```
[ ]: # Training and testing the model
start = datetime.now()
mt.multi_tasking()

print(f"Training and testing time: {datetime.now() - start}")

[ ]: # Getting the results
mt.get_results()
```